

Ajloun National University
Faculty of Information Technology
Graduation Project II Thesis Report

EcoGuard Home: Smart Protection & Energy Automation

Updated IEEE-Style Professional Graduation Project Report
Including DHT Environmental Sensing and Android APK Integration

Prepared by:
Student Name(s): _____
Student ID(s): _____
Supervisor: _____
Submission Year: 2026

Title Page

Table 1. Project title page details.

Field	Details
Project title	EcoGuard Home: Smart Protection & Energy Automation
Report type	Graduation Project II Thesis Report - updated version
Institution	Ajloun National University
Faculty	Faculty of Information Technology
Controller platform	Arduino Mega 2560
Communication interface	HC-05 Bluetooth serial link
Protection sensors	MQ gas sensor, flame sensor, and DHT temperature/humidity sensor
Mobile application	Android APK with live status, configurable command symbols, door/servo controls and emergency notifications
Power architecture	Arduino charger for the Mega board and LM2596 regulated 5 V bus for external modules
Submission year	2026

This updated report preserves and expands the original project documentation. The original report introduced the smart protection and energy automation prototype using Arduino Mega, HC-05 Bluetooth, gas and flame sensing, relays, servo control and an LM2596-based power layer. This revision adds DHT environmental sensing, a complete Android application workflow, continuous temperature and humidity display, configurable command symbols, local phone alarms and APK build validation.

The document is prepared in an IEEE-style academic format: numbered sections, formal tables, numbered figures, engineering requirements, traceability, validation cases, and numbered references. The format is intentionally thesis-oriented rather than a short two-column conference paper because the graduation project requires complete analysis, design, implementation and validation documentation.

Certificate

This is to certify that the graduation project thesis report entitled "EcoGuard Home: Smart Protection & Energy Automation" has been revised and extended to document the final prototype updates. The revised work describes the analysis, design, implementation and validation of a smartphone-controlled smart home prototype based on Arduino Mega, Bluetooth communication, relay-based appliance control, gas and flame safety sensing, DHT temperature and humidity sensing, servo-based door/ventilation movement, buzzer alarm, and an Android APK with live status, configurable controls and emergency notification behavior.

The report is submitted in partial fulfillment of the requirements for Graduation Project II in the Faculty of Information Technology.

Supervisor Signature: _____ Date: _____

Examiner Signature: _____ Date: _____

Abstract

EcoGuard Home is a smart protection and energy automation prototype designed to improve home safety, appliance convenience and controlled energy use through a low-cost embedded platform. The updated prototype uses an Arduino Mega 2560 controller, HC-05 Bluetooth module, 8-channel relay module, MQ gas sensor, flame sensor, DHT temperature and humidity sensor, servo-driven door/ventilation mechanism, buzzer alarm, LM2596 buck converter and dedicated Arduino charger. The firmware receives compact Bluetooth command symbols from an Android application and continuously publishes a structured status payload containing fire, gas, temperature, humidity, analog readings, smart-mode state, alarm state and relay states.

The Android application was extended from a basic control dashboard into a complete project interface. It now displays temperature and humidity clearly in the Live Status area, requests status updates periodically, provides explicit Door OPEN and Door CLOSE controls for the servo channel, renames relays 5, 6 and 7 as Load 1, Load 2 and Load 3, allows users to edit button names and ON/OFF command symbols, suppresses switch glitches during asynchronous state updates, and issues a phone notification, vibration and local alarm sound during gas or fire emergencies. The command symbols used by the app are documented and matched to the firmware switch-case implementation.

This report contributes a complete professional documentation package for the final prototype. It updates the system requirements, architecture, data flow, wiring, pin allocation, command protocol, status payload, Android interface, implementation logic, validation plan, bill of materials and appendices. The report follows IEEE-style writing conventions with formal sectioning, tables, figure captions and numbered references. Because the prototype is educational and not a certified safety device, final deployment requires calibration, qualified electrical supervision, certified enclosures and fail-safe engineering.

Keywords: Smart home, Arduino Mega, Bluetooth, HC-05, DHT sensor, temperature, humidity, gas sensor, flame sensor, Android APK, relay automation, emergency notification, LM2596, embedded safety system.

Acknowledgement

The project team acknowledges the academic supervisor and the Faculty of Information Technology for guidance during the design and evaluation of the EcoGuard Home prototype. The team also acknowledges the practical value of open technical documentation, microcontroller libraries and Android development tools used to build and verify the embedded and mobile portions of the system.

This revised version specifically documents the final improvements that were made after the Android application was successfully built and installed. The report therefore records the prototype as implemented: DHT temperature and humidity reporting, live mobile status, configurable command symbols, relay and servo command compatibility, emergency phone alarms and an APK-ready Android project.

Revision Statement and IEEE-Style Compliance

The first revised document was shorter than the original report. This version corrects that issue by expanding the report rather than compressing it. The structure keeps the original engineering content and adds final implementation evidence, mobile-application design, DHT integration, updated testing and appendices. The goal is not to increase page count artificially, but to provide a complete and traceable graduation-project document.

Table 2. IEEE-style compliance and revision quality checklist.

Area	Applied practice
Document structure	Cover, title page, certificate, abstract, acknowledgement, table of contents, lists, chapters, references and appendices are included.
IEEE-style references	Sources are cited using bracketed numerical references such as [1], [2] and listed at the end.
Tables and figures	Every table and figure is numbered and introduced in the surrounding text.
Technical traceability	Requirements, command symbols, pin mapping and test cases are connected.
Implementation evidence	Firmware and selected Android app logic are documented in appendices.
Professional limitations	Prototype safety limitations, calibration requirements and future work are stated explicitly.

Table of Contents

Abstract
Acknowledgement
Revision Statement and IEEE-Style Compliance
List of Figures
List of Tables
List of Symbols and Abbreviations
Chapter 1: Introduction
Chapter 2: Background and Existing Work
Chapter 3: System Analysis and Requirements
Chapter 4: System Design
Chapter 5: System Implementation
Chapter 6: Validation, Results and Discussion
Chapter 7: Conclusions and Future Work
References
Appendix A: Project Plan
Appendix B: Bill of Materials
Appendix C: Final Arduino Firmware
Appendix D: Android Application Implementation Notes
Appendix E: APK Build and Installation Guide
Appendix F: Poster Draft and Final Presentation Notes
Appendix G: Revision Traceability Matrix

List of Figures

Figure 1. Updated high-level architecture of EcoGuard Home.
Figure 2. Iterative engineering methodology used for the updated prototype.
Figure 3. System context diagram.
Figure 4. Use case model.
Figure 5. Updated hardware wiring schematic including DHT sensor.
Figure 6. Power distribution model.
Figure 7. Arduino Mega pin allocation.
Figure 8. Bluetooth command and status data flow.
Figure 9. Firmware state machine.
Figure 10. Command and emergency sequence.
Figure 11. Smart automation timing diagram.
Figure 11. Periodic sampling and status update schedule.
Figure 13. Updated Android application layout showing temperature, humidity and controls.
Figure 14. Representative sensor response.
Figure 15. Load-on time comparison.
Figure 16. Prototype current budget.
Figure 17. Validation coverage matrix.
Figure 18. Project Gantt chart.
Figure 19. Poster draft layout.

List of Tables

Table 1. Project title page details.
Table 2. IEEE-style compliance and revision quality checklist.
Table 3. Symbols and abbreviations.
Table 4. Updated hardware components.
Table 5. Updated software tools and libraries.
Table 6. Stakeholders and expected benefits.
Table 7. Functional requirements.
Table 8. Non-functional requirements.
Table 9. Arduino Mega pin allocation.
Table 10. Bluetooth command protocol.
Table 11. Status message dictionary.
Table 12. Android application default control configuration.
Table 13. Firmware module responsibilities.
Table 14. Android application module responsibilities.
Table 15. System test cases.
Table 16. Estimated current budget.
Table 17. Bill of materials.
Table 18. Revision traceability matrix.

List of Symbols and Abbreviations

Table 3. Symbols and abbreviations used in the updated report.

Symbol	Meaning
ADC	Analog-to-digital converter
APK	Android Package Kit installation file
AO/DO	Analog output / digital output of a sensor module
DHT	Digital humidity and temperature sensor family, such as DHT11 or DHT22
GPIO	General-purpose input/output
HC-05	Bluetooth serial module used for local phone-to-controller communication
IEEE	Institute of Electrical and Electronics Engineers; used here for reference and report style
LM2596	Step-down buck converter used for regulated 5 V external module bus
MQ	Metal-oxide gas sensor module family
PWM	Pulse-width modulation
RX/TX	Receive/transmit serial pins
SPP	Serial Port Profile used by classic Bluetooth serial modules
UART	Universal asynchronous receiver-transmitter

Chapter 1: Introduction

1.1 Project Overview

Smart home systems combine sensors, communication, control logic and actuators to improve the safety and convenience of residential environments. EcoGuard Home demonstrates this concept using accessible educational hardware and a custom Android interface. The system allows a user to switch loads through Bluetooth, run a timed energy automation sequence, monitor sensor values and receive emergency indications when gas or fire-related events are detected.

The final prototype is based on Arduino Mega 2560 because the project requires multiple digital and analog connections: eight relay outputs, HC-05 Bluetooth serial communication, gas and flame sensor inputs, DHT environmental sensing, a buzzer, and a servo actuator. Using the Mega avoids the pin limitations that would appear on smaller boards and leaves expansion capacity for future displays, logging modules or IoT connections.

The revised implementation adds a DHT sensor to measure temperature and humidity. These values are not used as primary fire detection decisions in the current safety logic; rather, they provide useful environmental monitoring information to the user and enrich the live dashboard. The app displays the readings as clear live-status cards and updates them whenever the controller sends a STATUS message.

The mobile application is now a major part of the system rather than a simple Bluetooth terminal. It lists paired HC-05 devices, opens Bluetooth settings, connects using the classic SPP UUID, sends command symbols, parses key-value status messages, displays live cards for safety and environmental data, supports door/servo controls, provides relay switches and stores custom button names and command symbols in local preferences.

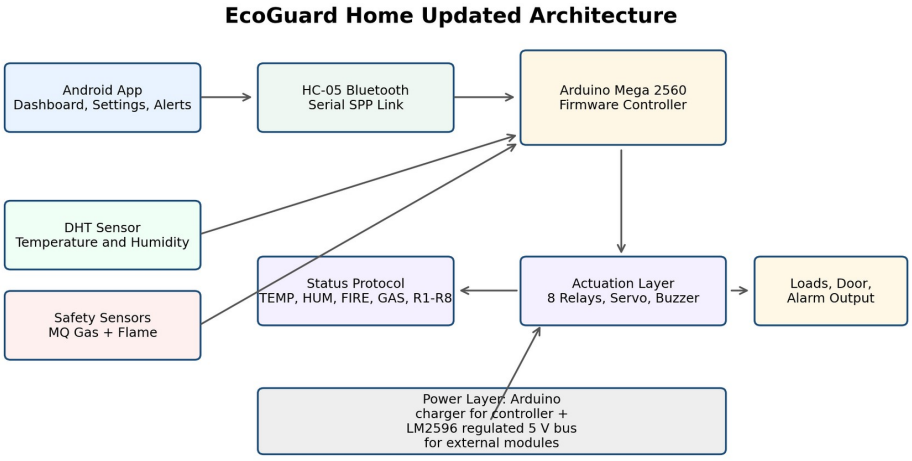


Figure 1. Updated high-level architecture of EcoGuard Home.

1.2 Problem Statement

Manual appliance control requires physical access to switches and provides no automatic response to unsafe events. A smart home prototype should therefore address convenience, energy-aware operation and safety response together. Without a unified interface, users may control loads from one tool, observe sensor values from another tool and miss critical alerts when a dangerous condition occurs.

The earlier project version focused on gas and flame protection, relay control and servo ventilation. After application testing, additional needs became clear. The user interface must display temperature and humidity from the DHT sensor, prevent relay switch glitches caused by asynchronous Bluetooth updates, match command symbols exactly with the Arduino firmware, and provide emergency notifications on the phone when FIRE or GAS becomes active.

Another problem is maintainability. In many Bluetooth relay projects, command symbols are hard-coded in the mobile application and become difficult to change if the firmware is updated. EcoGuard Home solves this problem by adding an application settings section that stores button labels and ON/OFF command symbols. This makes the app adaptable without requiring recompilation for every small command change.

1.3 Aim, Objectives and Contributions

The main aim is to design, implement and document a smartphone-controlled smart home prototype that integrates appliance control, energy automation, local safety response, environmental monitoring and Android-based user feedback.

- Design a modular hardware architecture using Arduino Mega, HC-05, relay module, MQ gas sensor, flame sensor, DHT sensor, servo motor, buzzer, LM2596 regulator and Arduino charger.
- Develop firmware that receives Bluetooth commands, reads sensors periodically, sends structured status messages and performs emergency shutdown.
- Create an Android APK that displays Fire, Gas, Temperature, Humidity, Smart Mode, Alarm, Door/Servo and relay states in a clear dashboard.
- Ensure command-symbol compatibility between the Android app and Arduino switch-case logic.
- Add configurable app settings for relay names and ON/OFF command symbols.
- Document validation procedures, safety limitations and future professional improvements using IEEE-style writing.

1.4 Scope and Limitations

The scope includes local Bluetooth control, relay switching, smart automation for relays 5, 6 and 7, gas and flame event detection, DHT environmental status reporting, servo door/ventilation movement, buzzer alarm activation, phone notifications and structured reporting. The design is intended for a laboratory prototype and graduation demonstration.

The system is not a certified household safety product. It should not be used as the only life-safety device. Gas thresholds require calibration, the flame sensor requires line-of-sight placement, the relay wiring must be supervised if mains loads are used, and production-level deployment would require certified enclosures, fuses, isolation, cable strain relief and fail-safe alarm latching.

Chapter 2: Background and Existing Work

2.1 Smart Home Automation Background

An embedded smart home controller receives commands from a user interface, reads sensors, applies decision logic and drives actuators. Educational prototypes often use Arduino boards because they support accessible programming, direct GPIO control, analog input, serial communication and timing functions. EcoGuard Home builds on this model while adding safety and mobile-status feedback.

Bluetooth-based control remains useful for local demonstrations because it avoids cloud accounts, router setup and Internet dependency. The trade-off is limited range, which is acceptable for a classroom prototype. A local safety response is also valuable because the Arduino can shut down loads and operate the buzzer even if the phone is disconnected.

2.2 Arduino Mega 2560 as Embedded Controller

Arduino Mega 2560 is appropriate because the prototype needs eight relay outputs, multiple sensors, a servo, a buzzer and a hardware serial port for the HC-05. The firmware uses Serial1 for Bluetooth while the USB serial port remains available for debugging through the Serial Monitor. This separation improves testing because status messages can be observed on both communication paths.

The final pin map is written explicitly in the design chapter. The DHT data signal is assigned to D23, while the servo remains on D22. This keeps the environmental sensor away from the relay pins D2-D9, the buzzer pin D11 and the Bluetooth pins D18/D19.

2.3 DHT Temperature and Humidity Sensing

The DHT sensor family provides digital temperature and relative-humidity readings through a single data line. In this project, either DHT11 or DHT22 can be used by changing the DHTTYPE definition in the firmware. The DHT readings are slower than ordinary digital GPIO reads; therefore, the firmware samples the DHT sensor every two seconds instead of every loop iteration.

The DHT sensor is used as an environmental monitoring feature. The readings are transmitted in the same STATUS payload as the gas, flame, smart-mode, alarm and relay fields. If the sensor returns an invalid reading, the firmware sends TEMP=ERR and HUM=ERR so that the app can display an error state instead of misleading numbers.

2.4 Android Application and Bluetooth Interface

The Android application communicates with HC-05 using a Bluetooth socket and the classic SPP UUID. After the user pairs the phone with HC-05 from Android settings, the app lists bonded devices, selects the likely HC-05 module, opens the socket and starts a reader thread. Incoming bytes are buffered until a line ending is received, then parsed as READY, STATUS, ALERT or SAFE messages.

The revised application includes local storage for control settings. Default names and command symbols match the Arduino firmware, but a settings dialog allows a user to edit button labels and the ON/OFF symbol for each relay. This is important for maintainability because the mobile application does not need code changes when a command symbol is intentionally changed during demonstration.

2.5 Related Work and Project Position

Table 4. Relationship between EcoGuard Home and related smart home directions.

Work or direction	Core idea	Relation to EcoGuard Home

Bluetooth relay automation	Phone sends simple commands to switch appliances.	EcoGuard adds sensor-driven emergency shutdown, DHT monitoring and settings.
IoT gas/fire monitoring	Sensors detect gas/fire and send alarms.	EcoGuard implements local response without relying on Internet connectivity.
Energy automation scheduling	Controller sequences loads according to timing or conditions.	EcoGuard demonstrates an 80 s schedule for relays 5, 6 and 7.
Mobile dashboard systems	App presents real-time data from embedded controller.	EcoGuard app displays fire, gas, temperature, humidity and relay states.
This project	Arduino Mega + Android APK + safety sensors + DHT + relays + servo.	Integrates mobile control, environmental monitoring and emergency response in one prototype.

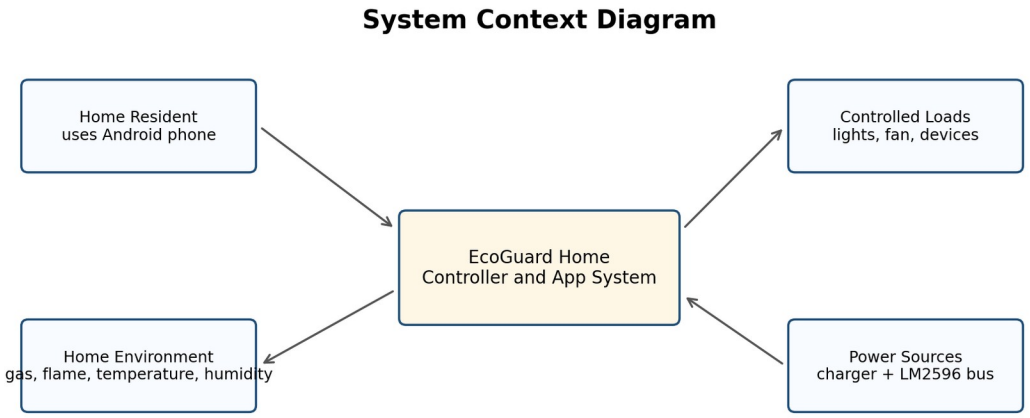


Figure 3. System context diagram for the updated EcoGuard Home system.

Chapter 3: System Analysis and Requirements

3.1 Stakeholders and Operating Scenario

The operating scenario assumes a laboratory smart-home model. The resident uses an Android phone to connect to HC-05, monitor the environment and control loads. The maintainer or student team calibrates thresholds, checks power wiring and verifies that the application command symbols match the firmware. The supervisor or examiner evaluates whether the system satisfies requirements and whether the documentation is traceable.

Table 5. Stakeholders and expected benefits.

Stakeholder	Expected benefit
Home resident	Convenient local phone control, live status, alarm notification and emergency shutdown.
Maintainer or student team	Clear wiring, configurable command symbols, repeatable tests and maintainable firmware.
Supervisor or examiner	Professional report, design artifacts, validation evidence and clear contribution.
Future developer	Expandable protocol for logging, IoT, calibration and improved safety behavior.

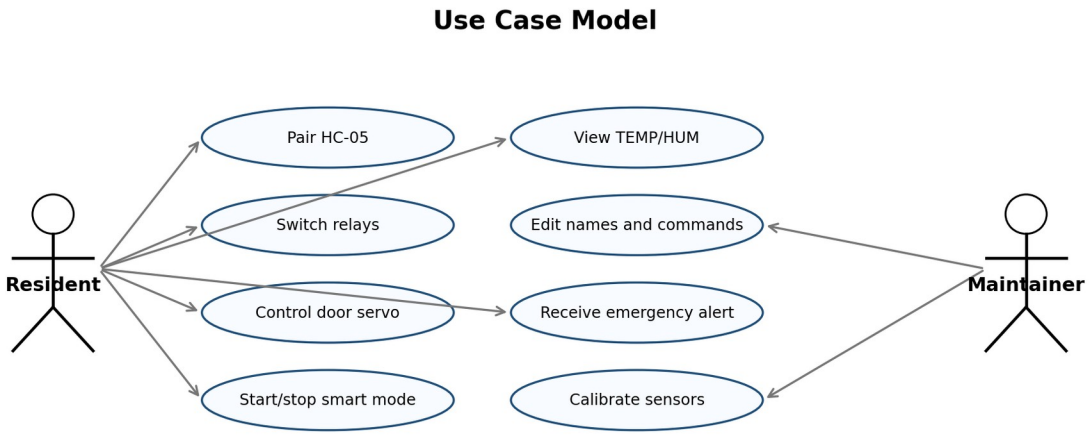


Figure 4. Use case model for the updated smart protection and automation prototype.

3.2 Functional Requirements

Table 6. Functional requirements for the updated EcoGuard Home prototype.

ID	Requirement	Verification method
FR-01	Receive Bluetooth commands through HC-05 Serial1.	Send every supported symbol from the app and observe firmware action.
FR-02	Switch eight relay channels with active-LOW logic.	Observe each relay LED or load state after ON/OFF commands.
FR-03	Run smart schedule with relays 5, 6 and 7.	Measure 0-80 s sequence using stopwatch or serial log.
FR-04	Control door/servo from the Android app.	Press Door OPEN and Door CLOSE; verify servo movement.
FR-05	Read MQ gas and flame sensors every 500 ms.	Observe GAS_ADC, FLAME_ADC and alert fields in STATUS messages.
FR-06	Read DHT temperature and humidity every 2000 ms.	Observe TEMP and HUM in the Android live status cards.
FR-07	Send STATUS payload every 1000 ms and on request.	Press Request Current Status and inspect system log.
FR-08	Trigger local and phone emergency response on FIRE=1 or GAS=1.	Simulate safe sensor trigger; check buzzer, shutdown, notification and alarm sound.
FR-09	Allow configurable button names and ON/OFF symbols.	Modify values in Settings, save, and confirm symbols sent.
FR-10	Prevent UI switch glitches caused by asynchronous status updates.	Toggle relays repeatedly and verify switches do not bounce incorrectly.

3.3 Non-Functional Requirements

Table 7. Non-functional requirements and measurable evidence.

Quality attribute	Requirement	Evidence or measurement
Safety	Default relay state shall be OFF at startup.	All relay pins are written HIGH in setup().
Responsiveness	Button action should be visible within one second in normal connection conditions.	App sends one-character commands; Arduino switch-case applies output directly.
Maintainability	Command labels and symbols should be configurable in the app.	Settings dialog saves values to SharedPreferences.
Power reliability	High-current modules should use external regulated 5 V bus.	LM2596 bus supplies relay module, servo and sensors; grounds are common.
Usability	Live environmental readings must be clear.	Temperature and Humidity cards are shown in Live Status.
Traceability	Firmware, app and report must use the same protocol.	Command protocol table lists every symbol and meaning.

3.4 Risk Analysis

Table 8. Main project risks and mitigation actions.

Risk	Impact	Likelihood	Mitigation
DHT invalid readings	Medium	Medium	Report ERR state, check wiring, use correct DHTTYPE and pull-up resistor.
Gas threshold not calibrated	High	Medium	Record baseline and alarm readings during final demo; adjust threshold.
Flame false trigger	Medium	Medium	Limit field of view and avoid strong infrared sources.
Bluetooth disconnection	Low for local safety	Medium	Arduino safety logic remains local; app displays disconnected state.
Relay or servo current spikes	Medium	Medium	Use LM2596 bus and common ground; avoid powering servo from board 5 V.
Command mismatch between app and firmware	High	Low after update	Defaults match firmware; settings allow controlled changes.
Phone notification permission not granted	Medium	Medium	Request notification permission and still update in-app alarm cards.

Chapter 4: System Design

4.1 Overall Architecture

The updated architecture is divided into six layers: mobile user interface, Bluetooth communication, embedded control, sensing, actuation and power. The Android app forms the user interface layer. The HC-05 module bridges the app to Arduino Serial1. The Arduino Mega firmware parses commands, samples sensors, controls outputs and publishes status. The sensing layer includes MQ gas, flame and DHT modules. The actuation layer includes relays, servo and buzzer. The power layer separates controller power from external module power.

One design decision is to keep emergency logic local on the Arduino. The phone enhances the emergency response by displaying alerts and sounding a local alarm, but the Arduino does not depend on the app to shut off relays or move the servo. This division is safer for a prototype because a lost Bluetooth connection does not disable the embedded safety response.

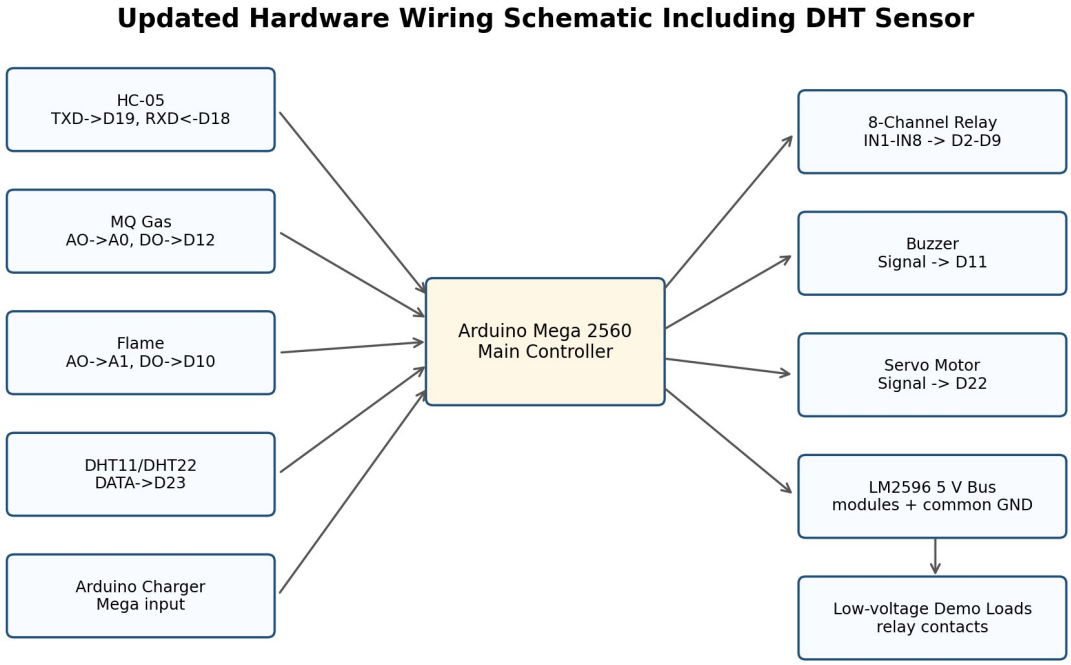


Figure 5. Updated hardware wiring schematic including DHT sensor.

4.2 Hardware Component Design

Table 9. Updated hardware components and design purposes.

Component	Role	Updated design purpose
Arduino Mega 2560	Main controller	Provides enough I/O and Serial1 Bluetooth port.
HC-05 Bluetooth module	Local wireless link	Receives command symbols and transmits status messages.
8-channel relay module	Load switching	Controls eight active-LOW relay channels.
MQ gas sensor	Gas leakage detection	Provides analog and digital gas signals.
Flame sensor	Fire/flame detection	Provides fire indication and optional analog reading.
DHT11/DHT22 sensor	Temperature and humidity sensing	Adds environmental monitoring for Android live status.
Servo motor	Door/ventilation actuator	Moves to open, close or emergency position.
Buzzer	Local audible alarm	Sounds during emergency interval.
LM2596 buck converter	External 5 V bus	Supplies relay, sensors, servo and modules.
Android phone	User interface	Runs APK for control, monitoring and emergency notification.

4.3 Electrical and Power Design

The relay board, MQ sensor heater and servo can draw more current than the Arduino regulator should supply. The updated design therefore uses an LM2596 buck converter adjusted to 5.0 V for external modules. The Arduino Mega is powered by its own charger. All low-voltage grounds must be connected to create a common signal reference.

The HC-05 RXD input should be protected with a resistor divider or level shifting if the breakout module does not include protection. Relay contacts must be wired according to the load voltage and current rating. For final demonstration, low-voltage loads are strongly recommended unless a qualified person supervises mains wiring.

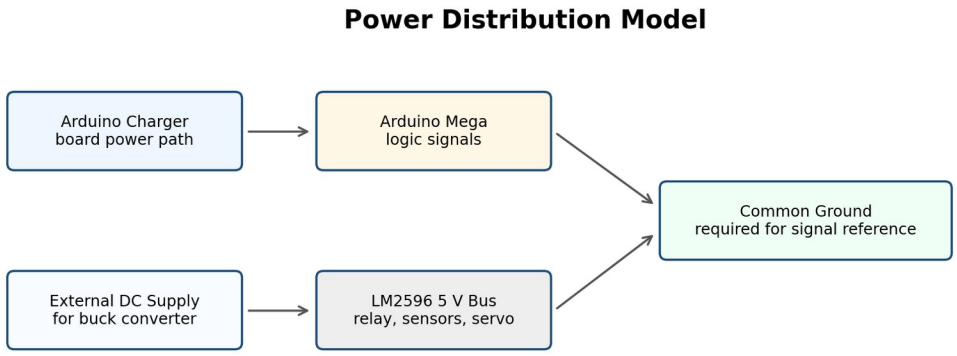


Figure 6. Power distribution model for the Arduino charger and LM2596 regulated bus.

4.4 Pin Allocation

Table 10. Revised Arduino Mega pin allocation including the DHT sensor.

Device signal	Mega pin	Direction	Updated note
Relay IN1-IN8	D2-D9	Output	Active-LOW relay inputs; HIGH=OFF, LOW=ON
Flame sensor DO	D10	Input	Fire alarm input with active-LOW module support
Buzzer	D11	Output	tone() output at 3000 Hz during alarm interval
MQ gas sensor DO	D12	Input	Gas digital alarm line with INPUT_PULLUP
HC-05 TXD	D19/RX1	Input	Bluetooth serial receive on Mega Serial1
HC-05 RXD	D18/TX1	Output	Bluetooth serial transmit; level protection recommended
Servo signal	D22	Output	Window/door servo signal pin
DHT data	D23	Input	Temperature/humidity sensor data input
MQ gas AO	A0	Input	Analog gas reading used with threshold
Flame AO	A1	Input	Analog flame reading retained for diagnostics
Common ground	GND	Reference	Mega, LM2596 bus and modules share reference

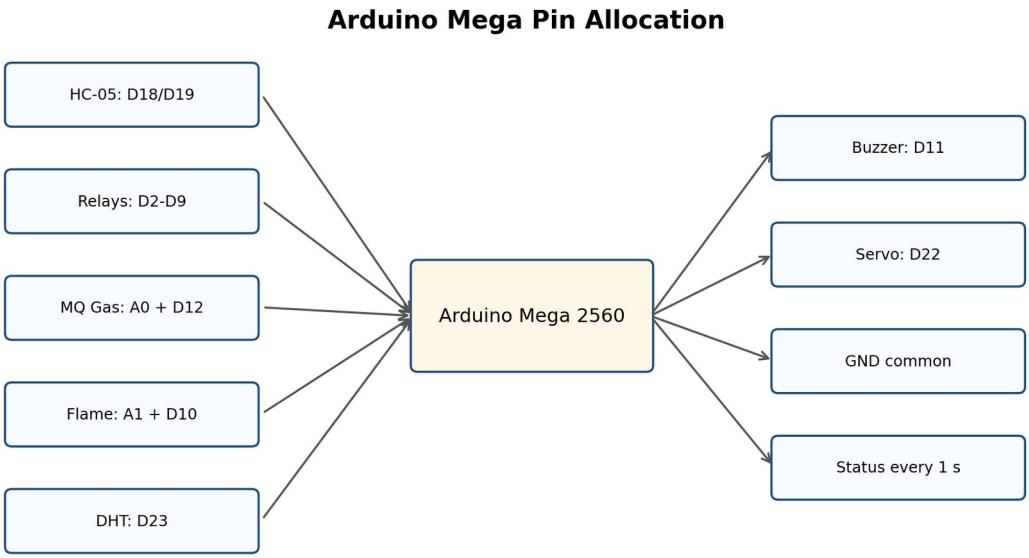


Figure 7. Arduino Mega pin allocation for Bluetooth, sensors and actuators.

4.5 Bluetooth Command Protocol

The command protocol uses single ASCII symbols. This makes the protocol simple to debug with a Bluetooth terminal and easy to send from buttons. The Android defaults exactly match the firmware. The settings screen may change labels and symbols, but any modified symbol must also match the Arduino switch-case logic if the firmware is changed.

Table 11. Bluetooth command protocol implemented in the firmware and application.

Command	Application action	Firmware effect
A	Smart ON	smartSystem=true; startTime=millis()
H	Smart OFF	Stop smart mode; relays 5, 6 and 7 OFF
L/P	CH1 Relay 1 ON/OFF	Manual state for relay index 0
R/T	Door/Servo OPEN/CLOSE and CH2 ON/OFF	Relay 2 state updates; servo 80 deg or 180 deg
Q/W	CH3 ON/OFF	Manual state for relay index 2
G/F	CH4 ON/OFF	Manual state for relay index 3
D/V	CH5 Load 1 ON/OFF	Manual state for relay index 4
E/N	CH6 Load 2 ON/OFF	Manual state for relay index 5
C/M	CH7 Load 3 ON/OFF	Manual state for relay index 6
I/O	CH8 Relay 8 ON/OFF	Manual state for relay index 7
Y	All Loads OFF	turnOffAllLoads() and smartSystem=false
X	Mute Alarm	alarmActive=false; noTone(buzzerPin)
S	Request Current Status	sendStatus()

4.6 Status Message Protocol

The controller sends structured messages that begin with STATUS, ALERT, SAFE or READY. The most important payload is STATUS because the application uses it to update live cards and relay switches. The revised payload includes the new DHT fields TEMP and HUM.

Example:
STATUS,FIRE=0,GAS=0,GAS_ADC=120,FLAME_ADC=850,TEMP=26.5,HUM=47.8,SMART=0,ALARM=0,R1=0,R2=0,R3=0,R4=0,R5=0,R6=0,R7=0,R8=0

Table 12. Status message dictionary used by the Android application.

Field	Meaning	Application behavior
FIRE	0 safe, 1 fire/flame detected	Updates Fire card and emergency notification logic.
GAS	0 safe, 1 gas detected	Updates Gas card and emergency notification logic.
GAS_ADC	Analog gas reading	Displayed in sensor readings row.
FLAME_ADC	Analog flame reading	Displayed in sensor readings row.
TEMP	Temperature in Celsius or ERR	Displayed on Temperature card.

HUM	Relative humidity percent or ERR	Displayed on Humidity card.
SMART	Smart mode flag	Updates Smart Mode card.
ALARM	Arduino alarm flag	Updates Alarm card.
R1-R8	Relay states as 0/1	Synchronizes relay switches without creating UI glitches.

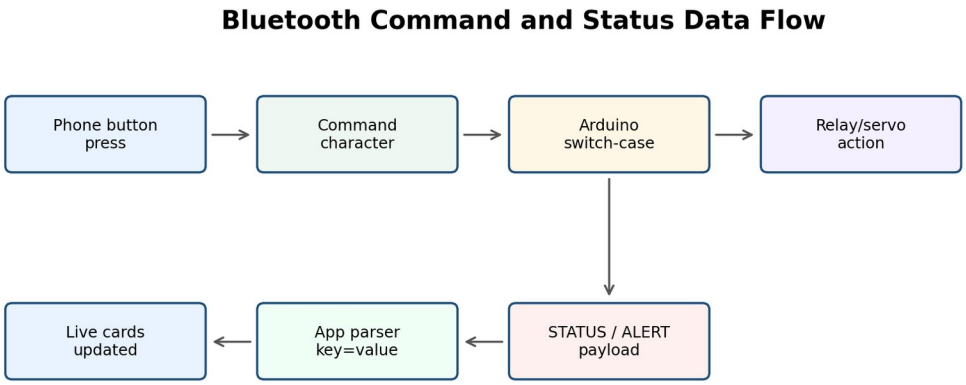


Figure 8. Bluetooth command and status data flow.

4.7 Android Application Interface Design

The app design is inspired by the project identity: a dark blue header, rounded white cards, green safe states, red/orange alarm states and clear labels. The Live Status area includes Bluetooth, Fire, Gas, Temperature, Humidity, Smart Mode, Alarm and Door/Servo cards. The Safety and Automation area includes Smart ON, Smart OFF, All Loads OFF, Mute Alarm and Request Current Status.

The Door/Servo Control section provides two explicit buttons: Door OPEN and Door CLOSE. These buttons use the ON/OFF command symbols configured for relay channel 2. The default mapping is R for open and T for close, matching the Arduino code that moves the servo to 80 degrees and 180 degrees respectively.

The Appliance Control section includes eight switches. Relay 5, Relay 6 and Relay 7 are renamed as Load 1, Load 2 and Load 3 in the application defaults. The Settings section allows the names and ON/OFF symbols to be changed from the phone. This design is useful during presentations because the team can adapt labels to the physical model without rebuilding the APK.

Table 13. Android application default control configuration.

Channel	Default app label	ON command	OFF command	Special note
CH1	CH1 - Relay 1	L	P	General relay channel.
CH2	CH2 - Door Open / Close	R	T	Also controls servo open/close.
CH3	CH3 - Relay 3	Q	W	General relay channel.
CH4	CH4 - Relay 4	G	F	General relay channel.
CH5	CH5 - Load 1	D	V	Smart-load channel 1.
CH6	CH6 - Load 2	E	N	Smart-load channel 2.
CH7	CH7 - Load 3	C	M	Smart-load channel 3.
CH8	CH8 - Relay 8	I	O	General relay channel.



Figure 9. Updated Android application layout showing temperature, humidity, door/servo controls and configurable load labels.

Chapter 5: System Implementation

5.1 Firmware Implementation

The firmware is written in Arduino C/C++. During setup, it starts USB serial and Bluetooth Serial1 at 9600 bps, initializes the DHT sensor, configures the relay pins as outputs, writes HIGH to all relay pins, configures gas and flame digital inputs with INPUT_PULLUP, attaches the servo and sends READY and STATUS messages. Because the relay board is active-LOW, logical ON is implemented by writing LOW.

The main loop reads Bluetooth commands, samples gas and flame sensors every 500 ms, samples the DHT sensor every 2000 ms, applies manual or smart-mode relay logic, manages the timed buzzer alarm and sends a STATUS message every 1000 ms. The use of millis() rather than delay() keeps the firmware responsive to Bluetooth commands and sensor events.

Table 14. Firmware module responsibilities after DHT integration.

Function	Responsibility	Key variables
setup()	Initializes serial ports, relays, inputs, DHT, buzzer and servo.	relayPins, dht, windowServo
loop()	Coordinates command reading, sampling, control and status sending.	lastSensorRead, lastDhtRead, lastStatusSent
readBluetoothCommands()	Parses one-character phone commands.	data, smartSystem
setManual()	Stores a requested manual relay state.	manualState[]

applyManualState()	Converts logical state to active-LOW relay output.	relayPins[]
checkGasAndFlame()	Reads MQ/flame channels and triggers emergency.	lastGasDetected, lastFlameDetected
readDHTSensor()	Reads temperature and humidity, validates NaN.	lastTemperature, lastHumidity, dhtValid
runSmartSystem()	Executes the 80 s relay schedule.	startTime, elapsedTime
sendStatus()	Publishes payload for app dashboard.	TEMP, HUM, R1-R8 fields

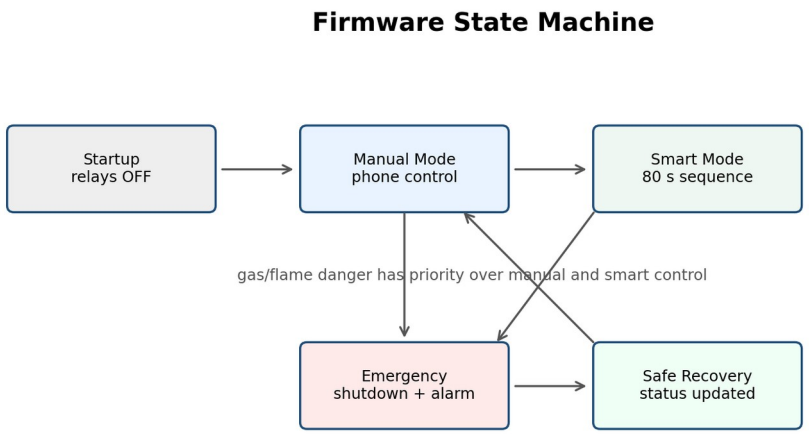


Figure 10. Firmware state machine for manual, smart and emergency modes.

5.2 DHT Firmware Integration

The DHT section adds the DHT library include, a DHT data pin definition, a DHTTYPE definition and a DHT object. The default is DHT11, but the line can be changed to DHT22 if that physical sensor is used. The two variables lastTemperature and lastHumidity store the most recent valid values, while dhtValid tells the status generator whether to print numeric values or ERR.

The DHT reading interval is intentionally set to two seconds. This avoids unnecessary reads and follows the practical behavior of DHT sensors, which are not intended to be polled hundreds of times per second. The app can still update continuously because the last valid reading is included in every STATUS message sent each second.

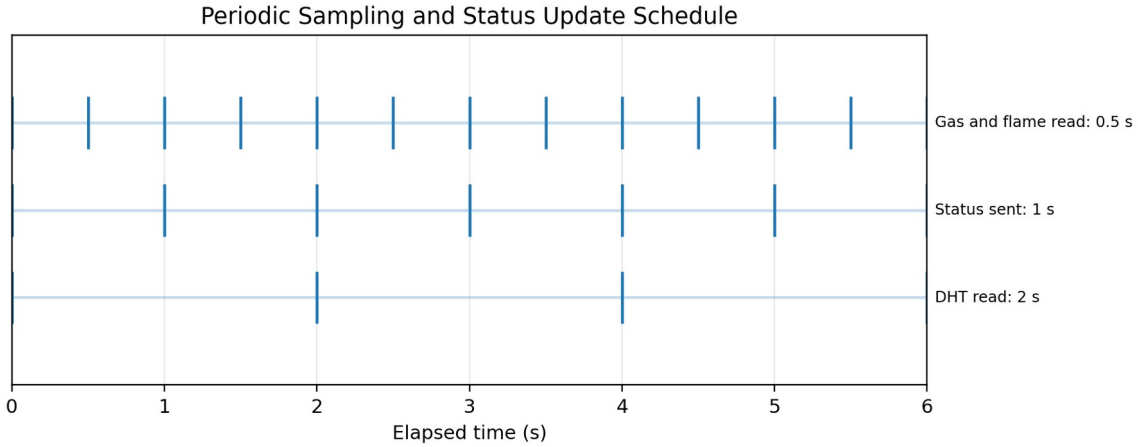


Figure 11. Periodic sampling and status update schedule.

5.3 Smart Schedule and Emergency Logic

The smart schedule begins when command A is received. During the first 20 seconds, relays 5, 6 and 7 remain OFF. From 20 to 40 seconds, relay 7 is ON. From 40 to 60 seconds, relay 7 and relay 5 are ON. From

60 to 80 seconds, relay 6 is ON. After 80 seconds, smart mode stops and the scheduled relays are forced OFF.

Emergency logic has higher priority than convenience logic. When gas or flame danger is detected for the first time, the firmware turns off all loads, moves the servo to a safe angle, activates the buzzer and sends an ALERT message. When the danger signal clears, the firmware resets the internal danger latch and sends SAFE. The current alarm interval is time-limited; a professional safety product should use a latched alarm state until inspection and manual reset.

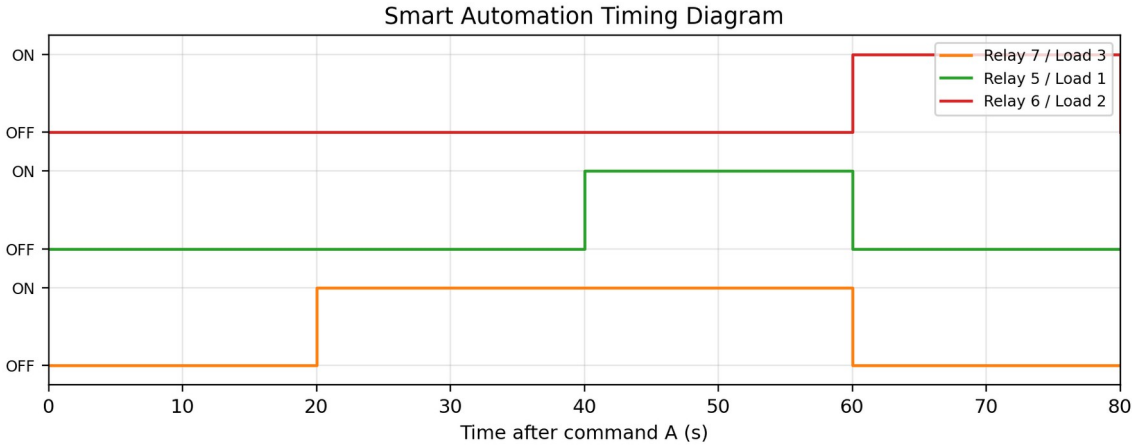


Figure 12. Smart automation timing diagram for relays 5, 6 and 7.

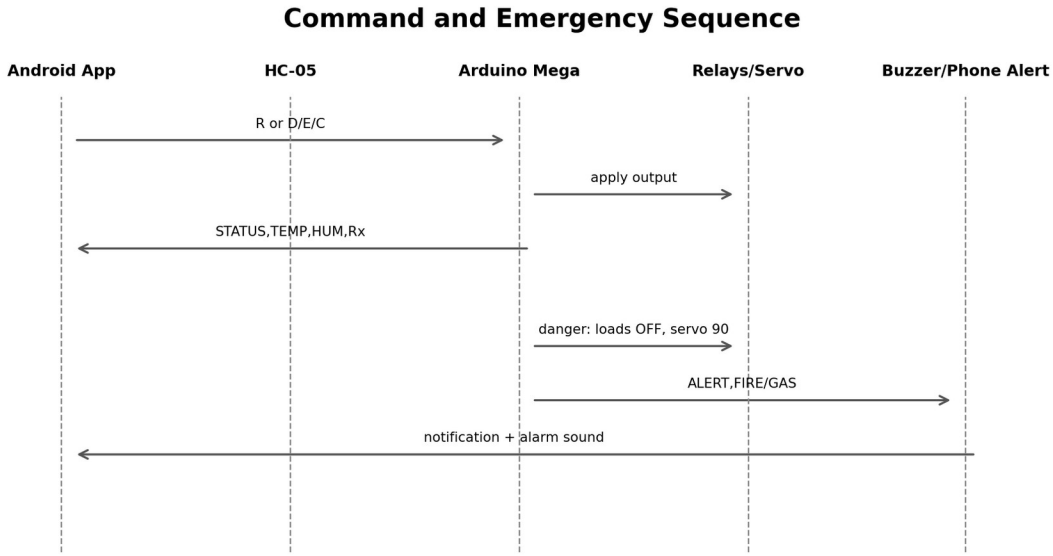


Figure 13. Command and emergency response sequence.

5.4 Android Application Implementation

The Android app is implemented as a native Android project. It builds the interface programmatically using Android views, stores button names and command symbols in SharedPreferences, communicates with the HC-05 through a BluetoothSocket and runs a reader thread to process incoming serial messages. The application dashboard follows the EcoGuard Home visual identity with dark blue branding, rounded cards and safety colors.

The app prevents the relay switch glitch observed in earlier testing by separating local user actions from remote status synchronization. When a switch is pressed, the app sends the configured command, stores the

pending state for a short period, disables the switch briefly and requests a fresh STATUS message. During the pending window, older remote values are ignored so that the switch does not bounce back incorrectly.

Incoming status messages are parsed as comma- or semicolon-separated key-value fields. The parser recognizes TEMP, TEMPERATURE and DHT_TEMP as temperature keys, and HUM, HUMIDITY and DHT_HUM as humidity keys. This makes the app tolerant to future firmware naming changes while keeping the default firmware payload concise.

Table 15. Android application module responsibilities.

App module or method	Purpose
buildInterface()	Creates dashboard, status cards, buttons, relay switches, settings and log.
refreshPairedDevices()	Lists bonded Bluetooth devices and prefers HC-05/EcoGuard names.
connectSelectedDevice()	Opens SPP Bluetooth socket and starts reader thread.
sendCommand()	Writes ASCII command symbols to Bluetooth output stream.
parseStatusMessage()	Updates Fire, Gas, Temperature, Humidity, Smart, Alarm and relay UI.
applyRemoteRelayState()	Suppresses stale relay states during pending command period.
showControlSettingsDialog()	Allows editing of relay labels and ON/OFF command symbols.
triggerEmergencyAlert()	Starts phone notification, vibration and local alarm sound.

5.5 Emergency Notification Implementation

When FIRE=1, GAS=1 or an ALERT message is received, the app evaluates whether an emergency notification should be issued. To avoid repeated notification spam, it suppresses duplicate emergency notifications for a short interval unless the previous emergency state was inactive. The notification title distinguishes fire, gas and combined emergencies.

The app also vibrates the phone and plays the default alarm ringtone. The Mute Alarm button stops the local phone alarm and sends command X to the Arduino. This gives the user a practical way to silence the alarm after acknowledging the condition while still leaving the system state visible on the dashboard.

5.6 APK Build Status

The project was built successfully in Android Studio using the debug APK workflow. During testing, an earlier installation showed the default "Hello Android" message because the wrong project directory was opened. After opening the correct EcoGuardHome_DHT_APKReady project, the application built and ran with the intended EcoGuard interface. This report records the corrected workflow so the same mistake is not repeated.

Chapter 6: Validation, Results and Discussion

6.1 Validation Strategy

Validation is divided into unit tests, integration tests, safety tests and APK tests. Unit tests verify individual relay channels, servo movement, DHT reading, gas/flame input logic and buzzer output. Integration tests verify Bluetooth command transfer, status parsing and app synchronization. Safety tests verify shutdown behavior and phone alarm behavior. APK tests verify that the correct Android project is built and installed.

Table 16. Updated system test cases and expected validation results.

ID	Test case	Procedure	Expected result
TC-01	Power-on initialization	Apply power to Mega charger and LM2596 bus.	All relays OFF, buzzer OFF, servo at 0 deg, app can request STATUS.
TC-02	Bluetooth connection	Pair phone with HC-05, refresh and connect.	Bluetooth card changes to Connected and system log shows data.
TC-03	Command compatibility	Press each relay and quick-action control.	Transmitted symbols match firmware protocol exactly.
TC-04	DHT display	Connect DHT11/DHT22 to D23 and watch app.	Temperature and Humidity cards update continuously.
TC-05	Door/servo	Press Door OPEN then Door CLOSE.	Commands R and T are sent; servo positions change.
TC-06	Smart schedule	Press Smart ON and observe 80 s cycle.	Relays 5, 6 and 7 follow the timing diagram.
TC-07	Gas emergency	Trigger gas sensor safely or simulate GAS=1.	Loads OFF, servo 90 deg, buzzer ON, phone alert issued.

TC-08	Fire emergency	Trigger flame sensor safely or simulate FIRE=1.	Loads OFF, servo 90 deg, buzzer ON, phone alert issued.
TC-09	Mute alarm	Press Mute Alarm in the app.	Command X is sent; phone alarm stops and Arduino buzzer is muted.
TC-10	All loads off	Press All Loads OFF.	Command Y is sent; all switch states become OFF.
TC-11	Settings persistence	Rename relay and change command symbol, restart app.	SharedPreferences restore the saved values.
TC-12	APK build and installation	Run assembleDebug and install app-debug.apk.	Application opens as EcoGuard Home, not default Hello Android.

6.2 Representative Results

The sensor-response figure shows the reporting form that should be used during final demonstration. The gas ADC value should be logged before, during and after a safe calibration event. The flame alarm should be recorded as a digital event. Temperature and humidity can be logged every two seconds and displayed continuously through the app. Final measured values should replace the representative curves after laboratory testing.

The energy comparison is expressed as normalized load-on time. If three smart loads remain ON for the full 80-second demonstration cycle, the total load-on time is 240 load-seconds. Under the implemented schedule, the selected loads are ON for approximately 80 load-seconds. The actual energy saving depends on the wattage of the connected loads, but the timing logic demonstrates how automation can reduce unnecessary operation.

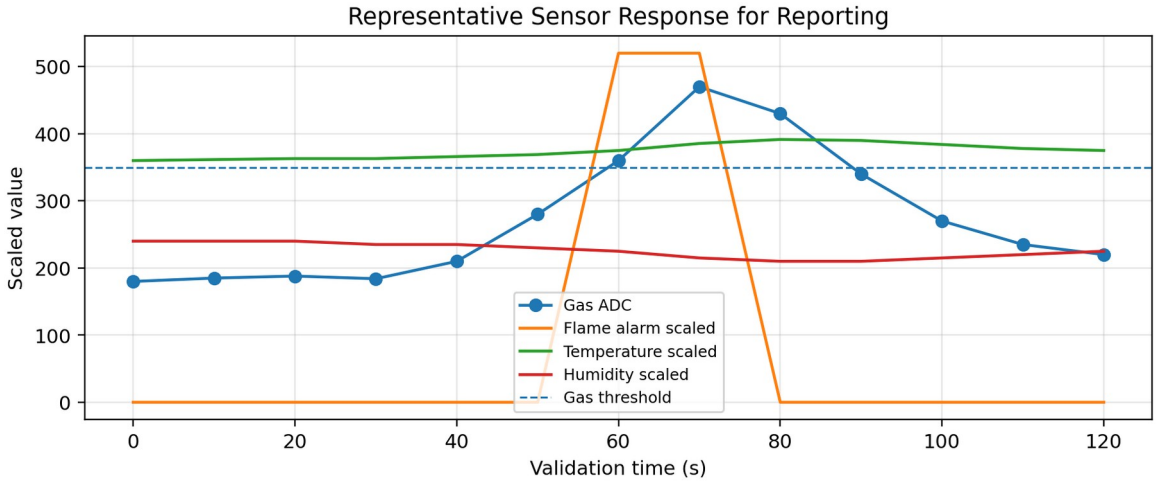


Figure 14. Representative gas, flame, temperature and humidity response for validation reporting.

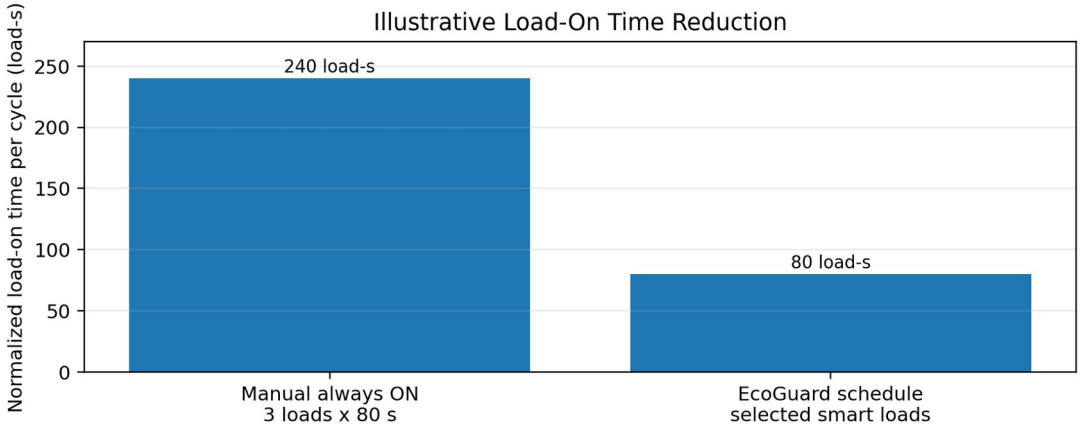


Figure 15. Illustrative normalized load-on time comparison for one 80 s cycle.

6.3 Power Budget Discussion

The current budget confirms why the LM2596 external 5 V bus is necessary. The relay board, MQ sensor heater and servo peak current may exceed what should be drawn from the Arduino board. The DHT sensor has low current demand, but it still shares the common ground and 5 V reference. The design target should provide enough margin for transient loads, especially servo movement during relay operation.

Table 17. Estimated current budget for the updated prototype.

Load	Estimated current (mA)	Design note
Arduino Mega 2560	80	Powered by dedicated charger; logic estimate.
HC-05 Bluetooth	35	Small serial module estimate.
MQ gas sensor	150	Heater-based module estimate.
Flame sensor	20	Small IR flame sensor module.
DHT sensor	2.5	Low-current environmental sensor.
8-channel relay module	560	Worst case if all channels energized.
Servo motor	650 peak	Use external 5 V bus; depends on load.
Buzzer	30	Small buzzer estimate.
Recommended bus target	>=2000	Provides margin for relay and servo transients.

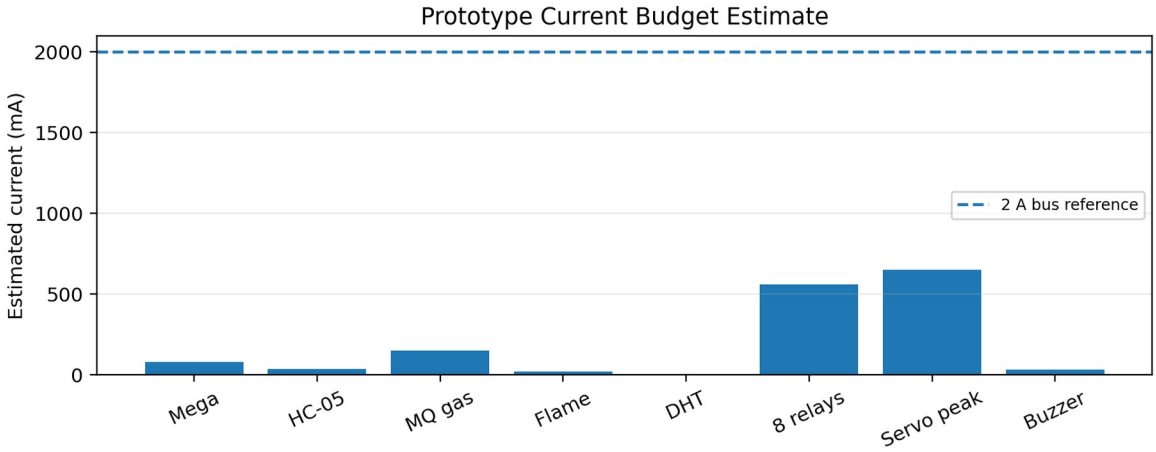


Figure 16. Prototype current budget estimate.

6.4 Validation Coverage

The validation coverage matrix summarizes planned coverage areas rather than certified safety performance. Each row represents a functional area that can be verified in a classroom demonstration. Passing these tests supports the educational prototype objective, but it does not replace calibration, certification or professional installation requirements.

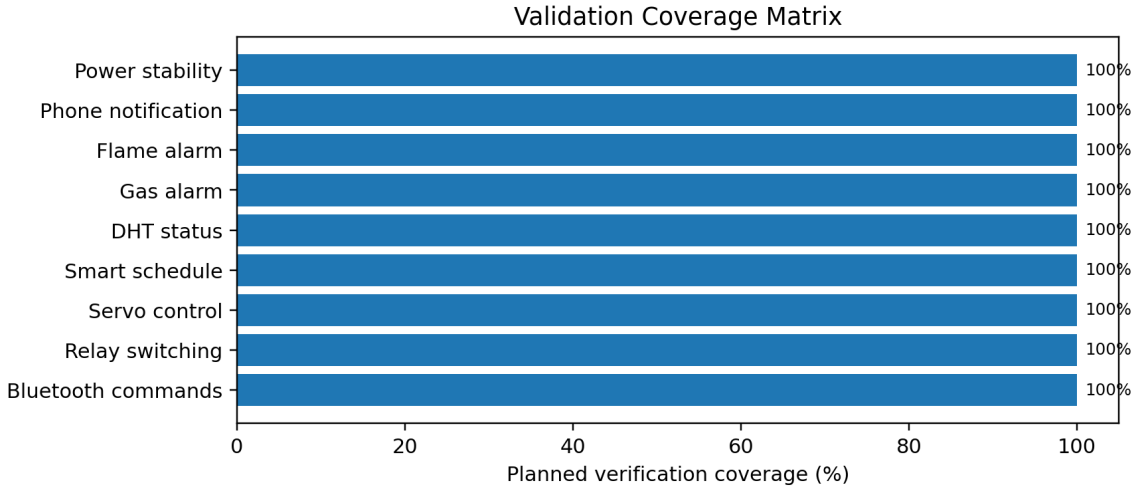


Figure 17. Validation coverage summary for the updated prototype.

6.5 Discussion

The most important improvement is the alignment between firmware, mobile application and report. The report now states the DHT pin, DHT update interval, TEMP/HUM status fields, relay command symbols and application control labels consistently. This consistency is essential for professional project documentation because mismatched diagrams, code and interface labels reduce credibility.

The Android settings feature improves maintainability but must be used carefully. If a command symbol is changed in the app without changing the Arduino firmware, the command will not produce the intended action. Therefore, the settings screen is best viewed as a controlled configuration tool for matching the app to a known firmware version.

The emergency notification feature improves user awareness, but the embedded controller remains responsible for local safety action. This is the correct architecture for a Bluetooth-based prototype because Bluetooth range, pairing state and phone permissions cannot be guaranteed at all times.

Chapter 7: Conclusions and Future Work

7.1 Conclusion

EcoGuard Home demonstrates how a low-cost embedded platform and Android application can combine smart home convenience, environmental monitoring, energy automation and local safety response. The updated system uses Arduino Mega, HC-05 Bluetooth, an 8-channel relay module, MQ gas sensor, flame sensor, DHT temperature/humidity sensor, servo actuator, buzzer, LM2596 external 5 V bus and a dedicated Arduino charger. The Android APK provides live dashboard cards, relay control, door/servo control, configurable labels and command symbols, system logging and emergency notifications.

The project achieves its educational objective by connecting hardware design, firmware implementation, mobile application development, command-protocol design, power planning and validation. The final report documents these elements in a professional IEEE-style format and expands the original report rather than reducing its detail.

7.2 Limitations

- The prototype is not a certified household safety device and should not be used as the only life-safety system.
- Gas thresholds must be calibrated in the final physical environment.
- Flame sensing requires line-of-sight and may be affected by strong infrared sources.
- Bluetooth control is local-range only and does not provide cloud monitoring.
- Phone notifications depend on Android permissions and user settings.
- Relay wiring for mains voltage requires qualified supervision, certified enclosure and proper protection.

7.3 Future Work

- Add ESP32 or ESP8266 IoT connectivity for remote cloud alerts and dashboards.
- Add persistent event logging to SD card, EEPROM or a mobile/cloud database.
- Add calibration mode in the Android app to adjust gas and flame thresholds safely.
- Add OLED or LCD local display for temperature, humidity, gas level and alarm state.
- Implement a latched emergency state that requires inspection and manual reset.
- Add battery backup for sensors and controller during power interruption.
- Use certified relay modules, fuses, terminal blocks and an enclosure for safer load wiring.
- Add authentication and pairing controls to reduce unauthorized command transmission.

References

[1] IEEE Author Center, "IEEE article templates and authoring resources," IEEE, accessed 2026.

[2] IEEE, "IEEE Editorial Style Manual for Authors," IEEE, accessed 2026.

[3] Arduino, "Arduino Mega 2560 Rev3 documentation," Arduino, accessed 2026.

[4] Arduino, "Arduino language reference," Arduino, accessed 2026.

[5] Arduino Libraries, "Servo library for Arduino," Arduino open-source library documentation, accessed 2026.

[6] Adafruit, "DHT sensor library for Arduino," open-source library documentation, accessed 2026.

[7] Aosong Electronics, "DHT11 humidity and temperature sensor datasheet," component datasheet.

[8] Aosong Electronics, "DHT22/AM2302 humidity and temperature sensor datasheet," component datasheet.

[9] Texas Instruments, "LM2596 SIMPLE SWITCHER step-down regulator datasheet," product documentation.

[10] Android Developers, "Bluetooth permissions," Android developer documentation, accessed 2026.

[11] Android Developers, "Notification runtime permission," Android developer documentation, accessed 2026.

[12] A. E. Amoran, A. S. Oluwole, E. O. Fagorola, and R. S. Diarah, "Home automated system using Bluetooth and an android application," Scientific African, vol. 11, 2021.

[13] M. A. Al Mamun, M. A. Hossain, M. A. Rahman, M. I. Abdullah, and M. S. Hossain, "Smart Home Automation System using Arduino and Android Application," Journal of Computer Science Engineering and Software Testing, vol. 6, no. 2, 2020.

[14] A.-N. Sharkawy, M. Hasanin, M. Sharf, M. Mohamed, and A. Elsheikh, "Development of Smart Home Applications Based on Arduino and Android Platforms: An Experimental Work," Automation, vol. 3, no. 4, pp. 579-595, 2022.

[15] G. Z. Islam et al., "IoT-Based Intelligent Gas Leakage Detection and Fire Protection System," International Journal of Interactive Mobile Technologies, vol. 16, no. 21, pp. 49-70, 2022.

[16] Project Team, "EcoGuard Home firmware, Android application and revised hardware design notes," internal graduation project materials, 2026.

Appendix A: Project Plan

The project plan was updated to include DHT integration, Android APK development and report revision. The final period focuses on integration testing, calibration, APK installation, demonstration preparation and final documentation.

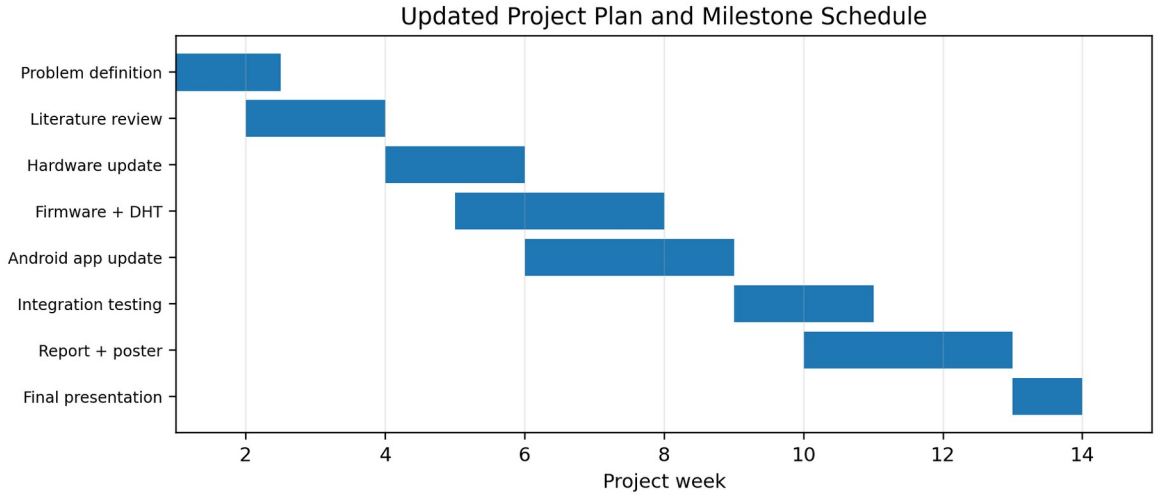


Figure 18. Updated project Gantt chart and milestone schedule.

Table 18. Milestones, deliverables and team roles.

Task	Time frame	Deliverable	Responsible role
Problem definition	Week 1-2	Problem statement, objectives and scope	All team members
Background review	Week 2-4	Technology justification and related work	Documentation lead
Hardware update	Week 4-6	Mega, sensors, DHT and power design	Hardware lead
Firmware + DHT	Week 5-8	DHT readings and updated STATUS protocol	Software lead
Android app update	Week 6-9	APK dashboard, settings and	Mobile lead

		notifications	
Integration testing	Week 9-11	Relay, servo, gas, flame, DHT and app tests	Testing lead
Report and poster	Week 10-13	IEEE-style report and presentation poster	Documentation lead
Final presentation	Week 14-15	Demonstration and viva preparation	All team members

Appendix B: Bill of Materials

Table 19. Bill of materials for the final updated prototype.

Item	Quantity	Purpose
Arduino Mega 2560	1	Main controller board
HC-05 Bluetooth module	1	Phone-to-controller serial interface
8-channel relay module	1	Switches controlled loads
MQ gas sensor module	1	Gas leakage detection
Flame sensor module	1	Fire/flame detection
DHT11 or DHT22 sensor	1	Temperature and humidity monitoring
Servo motor	1	Door/window/ventilation demonstration actuator
Buzzer	1	Audible alarm
LM2596 buck converter	1	Regulated 5 V external supply bus
Arduino charger/adaptor	1	Dedicated power input for Arduino Mega
External DC supply	1	Input source for LM2596
Jumper wires and terminal blocks	As needed	Signal and power connections
Low-voltage demonstration loads	As needed	Lights, fan or device models
Android phone	1	Runs EcoGuard Home APK

Appendix C: Final Arduino Firmware

The following firmware represents the final Arduino-side logic used by the updated application. It includes DHT temperature and humidity readings, the status payload required by the Android app, command compatibility for all relay channels, smart schedule logic and emergency response. If a DHT22 sensor is used instead of DHT11, only the DHTTYPE definition should be changed.

```
#include <Servo.h>
#include <DHT.h>

#define BT_SERIAL Serial1

const int relayPins[8] = {2, 3, 4, 5, 6, 7, 8, 9};

const int mqAnalogPin = A0;
const int mqDigitalPin = 12;

const int flameAnalogPin = A1;
const int flameDigitalPin = 10;

const int buzzerPin = 11;
const int servoPin = 22;

// DHT sensor settings
const int dhtPin = 23;
#define DHTTYPE DHT11   // Change to DHT22 if you use DHT22
DHT dht(dhtPin, DHTTYPE);

const int gasThreshold = 350;
const int flameThreshold = 300;

const bool gasActiveLow = true;
const bool flameActiveLow = true;

Servo windowServo;

bool smartSystem = false;
bool manualState[8] = {false, false, false, false, false, false, false, false};

bool alarmActive = false;
bool dangerAlreadyDetected = false;
bool lastGasDetected = false;
```

```
bool lastFlameDetected = false;

int lastGasValue = 0;
int lastFlameValue = 0;
int lastGasDigital = HIGH;
int lastFlameDigital = HIGH;

float lastTemperature = 0;
float lastHumidity = 0;
bool dhtValid = false;

unsigned long startTime = 0;
unsigned long alarmStartTime = 0;
unsigned long lastSensorRead = 0;
unsigned long lastStatusSent = 0;
unsigned long lastDhtRead = 0;

void setup() {
  Serial.begin(9600);
  BT_SERIAL.begin(9600);

  dht.begin();

  for (int i = 0; i < 8; i++) {
    pinMode(relayPins[i], OUTPUT);
    digitalWrite(relayPins[i], HIGH);
  }

  pinMode(mqDigitalPin, INPUT_PULLUP);
  pinMode(flameDigitalPin, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);
  noTone(buzzerPin);

  windowServo.attach(servoPin);
  windowServo.write(0);

  sendReady();
  sendStatus();
}

void loop() {
  readBluetoothCommands();

  if (millis() - lastSensorRead >= 500) {
    lastSensorRead = millis();
    checkGasAndFlame();
  }

  if (millis() - lastDhtRead >= 2000) {
    lastDhtRead = millis();
    readDHTSensor();
  }

  if (smartSystem) {
    runSmartSystem();
  } else {
    for (int i = 0; i < 8; i++) {
      applyManualState(i);
    }
  }

  runAlarm();

  if (millis() - lastStatusSent >= 1000) {
    lastStatusSent = millis();
    sendStatus();
  }
}

void readBluetoothCommands() {
  while (BT_SERIAL.available()) {
    char data = BT_SERIAL.read();
    if (data == '\n' || data == '\r') continue;

    switch (data) {
      case 'A':
        smartSystem = true;
        startTime = millis();
```

```
        sendStatus();
        break;

      case 'H':
        smartSystem = false;
        manualState[4] = false;
        manualState[5] = false;
        manualState[6] = false;
        digitalWrite(relayPins[4], HIGH);
        digitalWrite(relayPins[5], HIGH);
        digitalWrite(relayPins[6], HIGH);
        sendStatus();
        break;

      case 'L': setManual(0, true);  sendStatus(); break;
      case 'P': setManual(0, false); sendStatus(); break;

      case 'R':
        setManual(1, true);
        windowServo.write(80);
        sendStatus();
        break;

      case 'T':
        setManual(1, false);
        windowServo.write(180);
        sendStatus();
        break;

      case 'Q': setManual(2, true);  sendStatus(); break;
      case 'W': setManual(2, false); sendStatus(); break;
      case 'G': setManual(3, true);  sendStatus(); break;
      case 'F': setManual(3, false); sendStatus(); break;
      case 'D': setManual(4, true);  sendStatus(); break;
      case 'V': setManual(4, false); sendStatus(); break;
      case 'E': setManual(5, true);  sendStatus(); break;
      case 'N': setManual(5, false); sendStatus(); break;
      case 'C': setManual(6, true);  sendStatus(); break;
      case 'M': setManual(6, false); sendStatus(); break;
      case 'I': setManual(7, true);  sendStatus(); break;
      case 'O': setManual(7, false); sendStatus(); break;

      case 'Y':
        turnOffAllLoads();
        sendStatus();
        break;

      case 'X':
        alarmActive = false;
        noTone(buzzerPin);
        sendStatus();
        break;

      case 'S':
        sendStatus();
        break;
    }
  }
}

void setManual(int relayIndex, bool state) {
  if (relayIndex < 0 || relayIndex > 7) return;
  manualState[relayIndex] = state;

  if (!smartSystem) {
    applyManualState(relayIndex);
  }
}

void applyManualState(int relayIndex) {
  if (manualState[relayIndex]) {
    digitalWrite(relayPins[relayIndex], LOW);
  } else {
    digitalWrite(relayPins[relayIndex], HIGH);
  }
}

void turnOffAllLoads() {
  smartSystem = false;
```

```
  for (int i = 0; i < 8; i++) {
    manualState[i] = false;
    digitalWrite(relayPins[i], HIGH);
  }
}

void checkGasAndFlame() {
  lastGasValue = analogRead(mqAnalogPin);
  lastFlameValue = analogRead(flameAnalogPin);
  lastGasDigital = digitalRead(mqDigitalPin);
  lastFlameDigital = digitalRead(flameDigitalPin);

  bool digitalGasDetected = gasActiveLow ? (lastGasDigital == LOW) : (lastGasDigital == HIGH);
  bool digitalFlameDetected = flameActiveLow ? (lastFlameDigital == LOW) : (lastFlameDigital == HIGH);
  bool analogGasDetected = lastGasValue >= gasThreshold;
  bool analogFlameDetected = lastFlameValue <= flameThreshold;

  lastGasDetected = digitalGasDetected || analogGasDetected;
  lastFlameDetected = digitalFlameDetected || analogFlameDetected;

  bool dangerDetected = lastGasDetected || lastFlameDetected;

  if (dangerDetected && !dangerAlreadyDetected) {
    dangerAlreadyDetected = true;
    turnOffAllLoads();
    windowServo.write(90);
    alarmActive = true;
    alarmStartTime = millis();
    sendAlert();
  }

  if (!dangerDetected && dangerAlreadyDetected) {
    dangerAlreadyDetected = false;
    sendSafe();
  }
}

void readDHTSensor() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature(); // Celsius

  if (isnan(humidity) || isnan(temperature)) {
    dhtValid = false;
    return;
  }

  lastHumidity = humidity;
  lastTemperature = temperature;
  dhtValid = true;
}

void runAlarm() {
  if (alarmActive) {
    if (millis() - alarmStartTime < 20000) {
      tone(buzzerPin, 3000);
    } else {
      noTone(buzzerPin);
      alarmActive = false;
    }
  }
}

void runSmartSystem() {
  unsigned long elapsedTime = millis() - startTime;

  for (int i = 0; i < 8; i++) {
    applyManualState(i);
  }

  if (elapsedTime < 20000) {
    digitalWrite(relayPins[6], HIGH);
    digitalWrite(relayPins[4], HIGH);
    digitalWrite(relayPins[5], HIGH);
  } else if (elapsedTime < 40000) {
    digitalWrite(relayPins[6], LOW);
    digitalWrite(relayPins[4], HIGH);
    digitalWrite(relayPins[5], HIGH);
  } else if (elapsedTime < 60000) {
    digitalWrite(relayPins[6], LOW);
    digitalWrite(relayPins[4], LOW);
```

```
    digitalWrite(relayPins[5], HIGH);
  } else if (elapsedTime < 80000) {
    digitalWrite(relayPins[6], HIGH);
    digitalWrite(relayPins[4], HIGH);
    digitalWrite(relayPins[5], LOW);
  } else {
    smartSystem = false;
    manualState[4] = false;
    manualState[5] = false;
    manualState[6] = false;
    digitalWrite(relayPins[4], HIGH);
    digitalWrite(relayPins[5], HIGH);
    digitalWrite(relayPins[6], HIGH);
    sendStatus();
  }
}

void sendReady() {
  BT_SERIAL.println("READY,ECOGUARD_HOME");
  Serial.println("READY,ECOGUARD_HOME");
}

void sendSafe() {
  BT_SERIAL.print("SAFE");
  printStatusPayload(BT_SERIAL);
  BT_SERIAL.println();

  Serial.print("SAFE");
  printStatusPayload(Serial);
  Serial.println();
}

void sendAlert() {
  BT_SERIAL.print("ALERT");
  printStatusPayload(BT_SERIAL);
  BT_SERIAL.println();

  Serial.print("ALERT");
  printStatusPayload(Serial);
  Serial.println();
}

void sendStatus() {
  BT_SERIAL.print("STATUS");
  printStatusPayload(BT_SERIAL);
  BT_SERIAL.println();

  Serial.print("STATUS");
  printStatusPayload(Serial);
  Serial.println();
}

void printStatusPayload(Stream &port) {
  port.print(",FIRE=");
  port.print(lastFlameDetected ? 1 : 0);

  port.print(",GAS=");
  port.print(lastGasDetected ? 1 : 0);

  port.print(",GAS_ADC=");
  port.print(lastGasValue);

  port.print(",FLAME_ADC=");
  port.print(lastFlameValue);

  port.print(",TEMP=");
  if (dhtValid) {
    port.print(lastTemperature, 1);
  } else {
    port.print("ERR");
  }

  port.print(",HUM=");
  if (dhtValid) {
    port.print(lastHumidity, 1);
  } else {
    port.print("ERR");
  }
```

```
  port.print(",SMART=");
  port.print(smartSystem ? 1 : 0);

  port.print(",ALARM=");
  port.print(alarmActive ? 1 : 0);

  for (int i = 0; i < 8; i++) {
    port.print(",R");
    port.print(i + 1);
    port.print("=");
    port.print(digitalRead(relayPins[i]) == LOW ? 1 : 0);
  }
}
```

Appendix D: Android Application Implementation Notes

The Android project is contained in EcoGuardHome_DHT_APKReady. The important application file is MainActivity.java. It builds the interface, handles Bluetooth pairing and connection, writes command symbols, parses incoming status messages, suppresses relay switch glitches, stores settings and triggers emergency notifications.

Selected Android implementation excerpts are shown below to document the most relevant project logic without filling the thesis with the entire mobile source file.

```
private static final String[] DEFAULT_LABELS = {
            "CH1 - Relay 1",
            "CH2 - Door Open / Close",
            "CH3 - Relay 3",
            "CH4 - Relay 4",
            "CH5 - Load 1",
            "CH6 - Load 2",
            "CH7 - Load 3",
            "CH8 - Relay 8"
    };

    // These defaults exactly match the Arduino sketch sent earlier.
    private static final String[] DEFAULT_ON_COMMANDS = {"L", "R", "Q", "G", "D", "E", "C", "I"};
    private static final String[] DEFAULT_OFF_COMMANDS = {"P", "T", "W", "F", "V", "N", "M", "O"};

// --- excerpt break ---

private void sendRelayCommand(int index, boolean turnOn) {
        if (index < 0 || index >= relayStates.length) return;
        String command = turnOn ? relayOnCommands[index] : relayOffCommands[index];
        if (!sendCommand(command)) {
            setRelayUi(index, relayStates[index]);
            return;
        }

        relayStates[index] = turnOn;
        pendingRelayStates[index] = turnOn;
        pendingRelayUntil[index] = System.currentTimeMillis() + 900L;
        setRelayUi(index, turnOn);
        if (relaySwitches[index] != null) relaySwitches[index].setEnabled(false);
        final int relayIndex = index;
        mainHandler.postDelayed(() -> {
            if (relaySwitches[relayIndex] != null) relaySwitches[relayIndex].setEnabled(true);
            sendCommand("S", false);
        }, 300L);
    }

// --- excerpt break ---

private void parseStatusMessage(String message) {
        Map<String, String> map = parseKeyValueMessage(message);

        String fireValue = firstValue(map, "FIRE", "FLAME");
        String gasValue = firstValue(map, "GAS");
        String smartValue = firstValue(map, "SMART");
        String alarmValue = firstValue(map, "ALARM");
        String gasAdc = firstValue(map, "GAS_ADC", "GAS_VALUE");
        String flameAdc = firstValue(map, "FLAME_ADC", "FLAME_VALUE");
        String temperature = firstValue(map, "TEMP", "TEMPERATURE", "DHT_TEMP");
        String humidity = firstValue(map, "HUM", "HUMIDITY", "DHT_HUM");
```

```
        boolean hasFire = !fireValue.isEmpty();
        boolean hasGas = !gasValue.isEmpty();
        boolean fire = isOne(fireValue);
        boolean gas = isOne(gasValue);

        if (hasFire) updateFireStatus(fire);
        if (hasGas) updateGasStatus(gas);
        if (!smartValue.isEmpty()) setStatusCard(smartCard, "Smart Mode", isOne(smartValue) ? "ON" :
"OFF", isOne(smartValue) ? GREEN : MUTED);
        if (!alarmValue.isEmpty()) setAlarmUi(isOne(alarmValue));

        if (!gasAdc.isEmpty() || !flameAdc.isEmpty()) {
            String current = sensorReadings.getText().toString();
            String previousGas = extractDisplayedValue(current, "Gas ADC: ", "   |   Flame ADC:");
            String previousFlame = extractDisplayedValue(current, "Flame ADC: ", "");
            if (gasAdc.isEmpty()) gasAdc = previousGas.isEmpty() ? "--" : previousGas;
            if (flameAdc.isEmpty()) flameAdc = previousFlame.isEmpty() ? "--" : previousFlame;
            sensorReadings.setText("Gas ADC: " + gasAdc + "   |   Flame ADC: " + flameAdc);
        }

        if (!temperature.isEmpty()) updateTemperatureUi(temperature);
        if (!humidity.isEmpty()) updateHumidityUi(humidity);

        String relayString = firstValue(map, "RELAYS");
        if (!relayString.isEmpty()) {
            applyRelayString(relayString);
        } else {
            applyRelayValues(map);
        }

        if (message.startsWith("ALERT") || hasFire || hasGas) {
            evaluateEmergency(fire, gas, message);
        }
    }

// --- excerpt break ---

private void triggerEmergencyAlert(boolean fire, boolean gas, String rawMessage) {
        String title;
        String text;
        int notificationId;
        if (fire && gas) {
            title = "EcoGuard EMERGENCY";
            text = "Fire and gas danger detected. All loads should be OFF. Check the area immediately.";
            notificationId = 3003;
        } else if (fire) {
            title = "EcoGuard FIRE Alert";
            text = "Flame/fire sensor detected danger. Check the area immediately.";
            notificationId = 3001;
        } else {
            title = "EcoGuard GAS Alert";
            text = "Gas sensor detected abnormal level. Ventilate the area and check immediately.";
            notificationId = 3002;
        }
        toast(title);
        vibratePhone();
        playLocalAlarmSound();
        showNotification(title, text + "\n" + rawMessage, notificationId);
    }

// --- excerpt break ---

private void requestRequiredPermissions() {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.M) return;
        List<String> permissions = new ArrayList<>();
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            if (checkSelfPermission(Manifest.permission.BLUETOOTH_CONNECT) !=
PackageManager.PERMISSION_GRANTED) permissions.add(Manifest.permission.BLUETOOTH_CONNECT);
            if (checkSelfPermission(Manifest.permission.BLUETOOTH_SCAN) !=
PackageManager.PERMISSION_GRANTED) permissions.add(Manifest.permission.BLUETOOTH_SCAN);
        } else {
            if (checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) permissions.add(Manifest.permission.ACCESS_FINE_LOCATION);
        }
        if (Build.VERSION.SDK_INT >= 33 && checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS) !=
 PackageManager.PERMISSION_GRANTED) {
            permissions.add(Manifest.permission.POST_NOTIFICATIONS);
        }
        if (!permissions.isEmpty()) {
            requestPermissions(permissions.toArray(new String[0]), 2001);
        }
    }
```

Appendix E: APK Build and Installation Guide

Open Android Studio and select File > Open. Choose the EcoGuardHome_DHT_APKReady folder, not a new default MyApplication project. Wait for Gradle Sync to finish. Then use Build > Build Bundle(s) / APK(s) > Build APK(s). The debug APK should be located under app/build/outputs/apk/debug/app-debug.apk.

If the phone displays a default Hello Android screen, the wrong project was built or the old default app was installed. Delete the old app from the phone, open the correct EcoGuard project directory in Android Studio, rebuild the APK and install the new app-debug.apk.

Before connecting, pair the phone with HC-05 in Android Bluetooth settings. Common HC-05 pairing PIN values are 1234 or 0000. After pairing, open the EcoGuard app, press Refresh Devices, select HC-05 and press Connect. Then press Request Current Status to verify that STATUS messages are received.

- Confirm the app header reads EcoGuard Home.
- Confirm Temperature and Humidity cards are visible.
- Confirm Door OPEN sends R and Door CLOSE sends T by default.
- Confirm CH5, CH6 and CH7 are labeled Load 1, Load 2 and Load 3.
- Confirm Settings can edit button names and command symbols.
- Confirm emergency simulation changes cards to ALERT and produces phone notification/alarm.

Appendix F: Poster Draft and Final Presentation Notes

The poster should emphasize the final implemented features rather than only the hardware. Suggested sections are problem and aim, updated architecture, Android application, DHT integration, emergency response, validation results and future work. A QR code may link to a demonstration video or project repository if available.

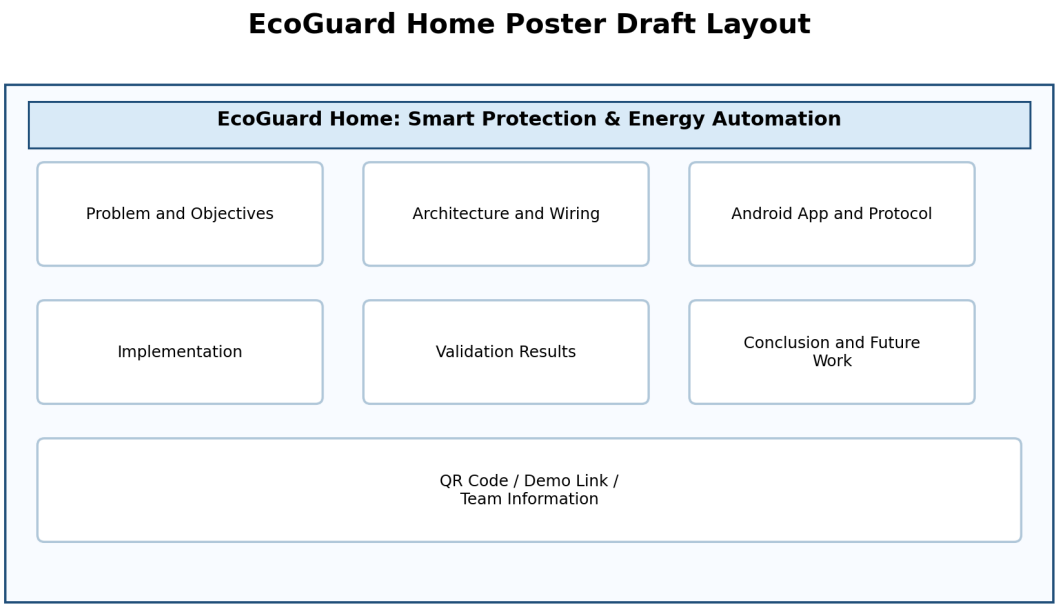


Figure 19. Recommended poster draft layout for the final presentation.

Appendix G: Revision Traceability Matrix

Table 20. Revision traceability matrix linking requested updates to report sections.

Requested update	Where addressed in report	Implementation evidence
DHT temperature and humidity display	Chapters 2, 4, 5 and Appendix C	TEMP/HUM fields, D23 pin, readDHTSensor().
Continuous update in app	Sections 4.6, 4.7 and 5.4	STATUS every 1 s, DHT every 2 s, live cards.
Command symbol compatibility	Sections 4.5 and 4.7	Command protocol and default app configuration.
Rename relays 5, 6 and 7	Section 4.7 and Table 13	CH5 Load 1, CH6 Load 2, CH7 Load 3.
Door open/close servo control	Sections 4.5, 4.7 and 5.3	Commands R/T and servo angles.
Reduce switch glitch	Section 5.4	Pending relay state logic and status refresh.
Editable button names and symbols	Sections 4.7 and 5.4	Settings dialog and SharedPreferences.
Phone alarm and emergency notification	Sections 5.5 and 6.1	Notification, vibration and ringtone behavior.
APK build documentation	Appendix E	Correct project opening and app-debug.apk path.
Professional IEEE-style report	Front matter, chapters and references	Numbered figures, tables, requirements and references.

Appendix H: Complete Android Application Source Extract

This appendix is included to keep the revised report complete and not compressed. The mobile application is part of the final engineering deliverable because it implements the live temperature/humidity cards, command-symbol compatibility, settings dialog, relay switch stabilization and emergency phone notification. The complete MainActivity.java source is listed so that the examiner can trace the Android behavior directly to the implemented APK.

H.1 MainActivity.java

```
package com.ecoguard.home;


import android.Manifest;
import android.annotation.SuppressLint;
import android.app.Activity;
import android.app.AlertDialog;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothSocket;
import android.content.Context;
import android.content.Intent;
import android.content.SharedPreferences;
import android.content.pm.PackageManager;
import android.graphics.Color;
import android.graphics.Typeface;
import android.graphics.drawable.GradientDrawable;
import android.media.AudioAttributes;
import android.media.Ringtone;
import android.media.RingtoneManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.os.VibrationEffect;
import android.os.Vibrator;
import android.provider.Settings;
import android.text.InputFilter;
import android.text.InputType;
import android.view.Gravity;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.GridLayout;
import android.widget.LinearLayout;
import android.widget.ScrollView;
import android.widget.Spinner;
import android.widget.Switch;
import android.widget.TextView;
import android.widget.Toast;


import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.UUID;


public class MainActivity extends Activity {


    private static final String APP_TITLE = "EcoGuard Home";
    private static final String CHANNEL_ID = "ecoguard_emergency_alerts";
    private static final String PREFS_NAME = "ecoguard_control_settings";
    private static final UUID SPP_UUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");


    private static final String[] DEFAULT_LABELS = {
            "CH1 - Relay 1",
            "CH2 - Door Open / Close",
            "CH3 - Relay 3",
            "CH4 - Relay 4",
            "CH5 - Load 1",
            "CH6 - Load 2",
            "CH7 - Load 3",
            "CH8 - Relay 8"
    };


    // These defaults exactly match the Arduino sketch sent earlier.
    private static final String[] DEFAULT_ON_COMMANDS = {"L", "R", "Q", "G", "D", "E", "C", "I"};
    private static final String[] DEFAULT_OFF_COMMANDS = {"P", "T", "W", "F", "V", "N", "M", "O"};


    private final int BLUE = Color.rgb(11, 61, 121);
    private final int BLUE_DARK = Color.rgb(6, 43, 86);
    private final int GREEN = Color.rgb(67, 160, 71);
    private final int RED = Color.rgb(211, 47, 47);
    private final int ORANGE = Color.rgb(245, 124, 0);
    private final int TEAL = Color.rgb(0, 137, 123);
    private final int SURFACE = Color.rgb(246, 249, 252);
    private final int CARD = Color.WHITE;
    private final int TEXT = Color.rgb(16, 32, 51);
    private final int MUTED = Color.rgb(96, 112, 128);


    private final Handler mainHandler = new Handler(Looper.getMainLooper());
    private final Object writeLock = new Object();


    private BluetoothAdapter bluetoothAdapter;
    private final ArrayList<BluetoothDevice> pairedDevices = new ArrayList<>();
```

```
private ArrayAdapter<String> deviceAdapter;
private Spinner deviceSpinner;
private Button connectButton;


private BluetoothSocket bluetoothSocket;
private InputStream inputStream;
private OutputStream outputStream;
private Thread readerThread;
private volatile boolean keepReading = false;
private volatile boolean connected = false;


private TextView bluetoothCard;
private TextView fireCard;
private TextView gasCard;
private TextView temperatureCard;
private TextView humidityCard;
private TextView smartCard;
private TextView alarmCard;
private TextView doorCard;
private TextView sensorReadings;
private TextView lastMessage;
private Button doorOpenButton;
private Button doorCloseButton;


private final Switch[] relaySwitches = new Switch[8];
private final boolean[] relayStates = new boolean[8];
private final boolean[] pendingRelayStates = new boolean[8];
private final long[] pendingRelayUntil = new long[8];
private final String[] relayLabels = new String[8];
private final String[] relayOnCommands = new String[8];
private final String[] relayOffCommands = new String[8];


private boolean updatingSwitches = false;
private boolean emergencyActive = false;
private long lastEmergencyNotificationTime = 0L;
private Ringtone localAlarmRingtone;
private final ArrayList<String> logLines = new ArrayList<>();


private final Runnable statusPollRunnable = new Runnable() {
    @Override
    public void run() {
        if (connected) {
            sendCommand("S", false);
            mainHandler.postDelayed(this, 3000L);
        }
    }
};


@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);


    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
        getWindow().setStatusBarColor(BLUE_DARK);
        getWindow().setNavigationBarColor(BLUE_DARK);
    }


    bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();
    loadControlSettings();
    createNotificationChannel();
    requestRequiredPermissions();
    buildInterface();
    refreshPairedDevices();
}


private void buildInterface() {
    ScrollView scrollView = new ScrollView(this);
    scrollView.setFillViewport(true);
    scrollView.setBackgroundColor(SURFACE);


    LinearLayout root = new LinearLayout(this);
    root.setOrientation(LinearLayout.VERTICAL);
    root.setPadding(dp(16), dp(16), dp(16), dp(24));
    scrollView.addView(root);


    LinearLayout header = new LinearLayout(this);
    header.setOrientation(LinearLayout.VERTICAL);
    header.setPadding(dp(20), dp(22), dp(20), dp(22));
    header.setBackground(roundedGradient(BLUE_DARK, BLUE));
    root.addView(header, matchWrapMargins(0, 0, 0, 14));


    TextView title = new TextView(this);
    title.setText(APP_TITLE);
    title.setTextColor(Color.WHITE);
    title.setTextSize(31);
    title.setTypeface(Typeface.DEFAULT_BOLD);
    title.setGravity(Gravity.CENTER_HORIZONTAL);
    header.addView(title);


    TextView subtitle = new TextView(this);
    subtitle.setText("Smart Protection & Energy Automation");
    subtitle.setTextColor(Color.rgb(210, 235, 255));
    subtitle.setTextSize(15);
    subtitle.setGravity(Gravity.CENTER_HORIZONTAL);
    subtitle.setPadding(0, dp(6), 0, 0);
    header.addView(subtitle);


    TextView tagline = new TextView(this);
    tagline.setText("Monitor  |  Protect  |  Automate");
    tagline.setTextColor(Color.WHITE);
    tagline.setTextSize(13);
    tagline.setGravity(Gravity.CENTER_HORIZONTAL);
    tagline.setPadding(0, dp(12), 0, 0);
    header.addView(tagline);


    root.addView(sectionTitle("Connection"));
    LinearLayout connectionCard = cardContainer();
    root.addView(connectionCard, matchWrapMargins(0, 0, 0, 14));
```

```
TextView hint = new TextView(this);
hint.setText("Pair your phone with HC-05 first, then select it here. Common pairing PIN: 1234 or 0000.");
hint.setTextColor(MUTED);
hint.setTextSize(13);
hint.setPadding(0, 0, 0, dp(10));
connectionCard.addView(hint);


deviceSpinner = new Spinner(this);
deviceAdapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_dropdown_item, new ArrayList<String>());
deviceSpinner.setAdapter(deviceAdapter);
connectionCard.addView(deviceSpinner, new LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(48)));


LinearLayout connectionButtons = row();
connectionCard.addView(connectionButtons, matchWrapMargins(0, 10, 0, 0));
Button refreshButton = primaryButton("Refresh Devices", BLUE);
connectButton = primaryButton("Connect", GREEN);
connectionButtons.addView(refreshButton, weightedButtonParams(1));
connectionButtons.addView(connectButton, weightedButtonParams(1));
refreshButton.setOnClickListener(v -> refreshPairedDevices());
connectButton.setOnClickListener(v -> {
    if (connected) disconnect(); else connectSelectedDevice();
});


Button bluetoothSettings = outlineButton("Open Android Bluetooth Settings");
bluetoothSettings.setOnClickListener(v -> startActivity(new Intent(Settings.ACTION_BLUETOOTH_SETTINGS)));
connectionCard.addView(bluetoothSettings, matchWrapMargins(0, 10, 0, 0));


root.addView(sectionTitle("Live Status"));
GridLayout statusGrid = new GridLayout(this);
statusGrid.setColumnCount(2);
root.addView(statusGrid, matchWrapMargins(0, 0, 0, 14));


bluetoothCard = statusCard("Bluetooth", "Disconnected", BLUE);
fireCard = statusCard("Fire", "SAFE", GREEN);
gasCard = statusCard("Gas", "SAFE", GREEN);
temperatureCard = statusCard("Temperature", "-- °C", TEAL);
humidityCard = statusCard("Humidity", "-- %", TEAL);
smartCard = statusCard("Smart Mode", "OFF", MUTED);
alarmCard = statusCard("Alarm", "OFF", MUTED);
doorCard = statusCard("Door / Servo", "UNKNOWN", BLUE);
statusGrid.addView(bluetoothCard, gridParams());
statusGrid.addView(fireCard, gridParams());
statusGrid.addView(gasCard, gridParams());
statusGrid.addView(temperatureCard, gridParams());
statusGrid.addView(humidityCard, gridParams());
statusGrid.addView(smartCard, gridParams());
statusGrid.addView(alarmCard, gridParams());
statusGrid.addView(doorCard, gridParams());


sensorReadings = new TextView(this);
sensorReadings.setText("Gas ADC: --   |   Flame ADC: --");
sensorReadings.setTextColor(MUTED);
sensorReadings.setTextSize(13);
sensorReadings.setPadding(dp(10), dp(8), dp(10), dp(8));
sensorReadings.setBackground(roundedStroke(Color.WHITE, Color.rgb(221, 229, 238), dp(14)));
root.addView(sensorReadings, matchWrapMargins(0, 0, 0, 14));


root.addView(sectionTitle("Safety & Automation"));
LinearLayout quickCard = cardContainer();
root.addView(quickCard, matchWrapMargins(0, 0, 0, 14));
LinearLayout firstActionRow = row();
quickCard.addView(firstActionRow);
Button smartOn = primaryButton("Smart ON", GREEN);
Button smartOff = primaryButton("Smart OFF", BLUE);
firstActionRow.addView(smartOn, weightedButtonParams(1));
firstActionRow.addView(smartOff, weightedButtonParams(1));
smartOn.setOnClickListener(v -> sendCommand("A"));
smartOff.setOnClickListener(v -> sendCommand("H"));


LinearLayout secondActionRow = row();
quickCard.addView(secondActionRow, matchWrapMargins(0, 10, 0, 0));
Button allOff = primaryButton("All Loads OFF", ORANGE);
Button muteAlarm = primaryButton("Mute Alarm", RED);
secondActionRow.addView(allOff, weightedButtonParams(1));
secondActionRow.addView(muteAlarm, weightedButtonParams(1));
allOff.setOnClickListener(v -> turnAllLoadsOffFromPhone());
muteAlarm.setOnClickListener(v -> {
    stopLocalAlarmSound();
    sendCommand("X");
});


Button statusRequest = outlineButton("Request Current Status");
quickCard.addView(statusRequest, matchWrapMargins(0, 10, 0, 0));
statusRequest.setOnClickListener(v -> sendCommand("S"));


root.addView(sectionTitle("Door / Servo Control"));
LinearLayout doorControlCard = cardContainer();
root.addView(doorControlCard, matchWrapMargins(0, 0, 0, 14));
TextView doorHint = new TextView(this);
doorHint.setText("Door OPEN sends the ON command for CH2. Door CLOSE sends the OFF command for CH2.");
doorHint.setTextColor(MUTED);
doorHint.setTextSize(13);
doorHint.setPadding(0, 0, 0, dp(10));
doorControlCard.addView(doorHint);
LinearLayout doorRow = row();
doorControlCard.addView(doorRow);
doorOpenButton = primaryButton("Door OPEN", GREEN);
doorCloseButton = primaryButton("Door CLOSE", BLUE);
doorRow.addView(doorOpenButton, weightedButtonParams(1));
doorRow.addView(doorCloseButton, weightedButtonParams(1));
doorOpenButton.setOnClickListener(v -> sendRelayCommand(1, true));
doorCloseButton.setOnClickListener(v -> sendRelayCommand(1, false));
updateDoorButtons();


root.addView(sectionTitle("Appliance Control"));
LinearLayout relayCard = cardContainer();
root.addView(relayCard, matchWrapMargins(0, 0, 0, 14));
for (int i = 0; i < 8; i++) {
    addRelaySwitch(relayCard, i);
}
```

```
    root.addView(sectionTitle("Settings"));
    LinearLayout settingsCard = cardContainer();
    root.addView(settingsCard, matchWrapMargins(0, 0, 0, 14));
    TextView settingsHint = new TextView(this);
    settingsHint.setText("Edit button names and the exact ON/OFF command symbols sent to Arduino. Defaults match the current firmware.");
    settingsHint.setTextColor(MUTED);
    settingsHint.setTextSize(13);
    settingsHint.setPadding(0, 0, 0, dp(10));
    settingsCard.addView(settingsHint);
    Button controlSettings = outlineButton("Button Names & Command Symbols");
    controlSettings.setOnClickListener(v -> showControlSettingsDialog());
    settingsCard.addView(controlSettings, matchWrapMargins(0, 0, 0, 0));


    root.addView(sectionTitle("System Log"));
    lastMessage = new TextView(this);
    lastMessage.setText("Waiting for EcoGuard Home data...");
    lastMessage.setTextColor(MUTED);
    lastMessage.setTextSize(13);
    lastMessage.setPadding(dp(14), dp(12), dp(14), dp(12));
    lastMessage.setMinLines(4);
    lastMessage.setBackground(roundedStroke(Color.WHITE, Color.rgb(221, 229, 238), dp(14)));
    root.addView(lastMessage, matchWrapMargins(0, 0, 0, 0));


    setContentView(scrollView);
}


private void addRelaySwitch(LinearLayout parent, int index) {
    Switch relaySwitch = new Switch(this);
    relaySwitch.setTextColor(TEXT);
    relaySwitch.setTextSize(15);
    relaySwitch.setPadding(dp(8), dp(10), dp(8), dp(10));
    relaySwitch.setOnCheckedChangeListener((buttonView, isChecked) -> {
        setSwitchText(index, isChecked);
        if (!updatingSwitches) {
            sendRelayCommand(index, isChecked);
        }
    });
    relaySwitches[index] = relaySwitch;
    setRelayUi(index, false);
    parent.addView(relaySwitch, matchWrapMargins(0, 0, 0, 4));
}


private void setSwitchText(int index, boolean isOn) {
    if (index < 0 || index >= relaySwitches.length || relaySwitches[index] == null) return;
    relaySwitches[index].setText(relayLabels[index] + "   " + (isOn ? "ON" : "OFF"));
}


private void sendRelayCommand(int index, boolean turnOn) {
    if (index < 0 || index >= relayStates.length) return;
    String command = turnOn ? relayOnCommands[index] : relayOffCommands[index];
    if (!sendCommand(command)) {
        setRelayUi(index, relayStates[index]);
        return;
    }


    relayStates[index] = turnOn;
    pendingRelayStates[index] = turnOn;
    pendingRelayUntil[index] = System.currentTimeMillis() + 900L;
    setRelayUi(index, turnOn);
    if (relaySwitches[index] != null) relaySwitches[index].setEnabled(false);
    final int relayIndex = index;
    mainHandler.postDelayed(() -> {
        if (relaySwitches[relayIndex] != null) relaySwitches[relayIndex].setEnabled(true);
        sendCommand("S", false);
    }, 300L);
}


private void setRelayUi(int index, boolean isOn) {
    if (index < 0 || index >= relayStates.length) return;
    relayStates[index] = isOn;
    Switch relaySwitch = relaySwitches[index];
    if (relaySwitch != null) {
        updatingSwitches = true;
        relaySwitch.setChecked(isOn);
        setSwitchText(index, isOn);
        updatingSwitches = false;
    }
    if (index == 1) updateDoorUi();
}


private void updateDoorUi() {
    setStatusCard(doorCard, "Door / Servo", relayStates[1] ? "OPEN" : "CLOSED", relayStates[1] ? GREEN : BLUE);
}


private void updateDoorButtons() {
    if (doorOpenButton != null) doorOpenButton.setText("Door OPEN  (" + relayOnCommands[1] + ")");
    if (doorCloseButton != null) doorCloseButton.setText("Door CLOSE  (" + relayOffCommands[1] + ")");
}


private void turnAllLoadsOffFromPhone() {
    if (!sendCommand("Y")) return;
    long until = System.currentTimeMillis() + 1000L;
    for (int i = 0; i < relayStates.length; i++) {
        relayStates[i] = false;
        pendingRelayStates[i] = false;
        pendingRelayUntil[i] = until;
        setRelayUi(i, false);
    }
    mainHandler.postDelayed(() -> sendCommand("S", false), 350L);
}


@SuppressLint("MissingPermission")
private void refreshPairedDevices() {
    pairedDevices.clear();
    if (deviceAdapter != null) deviceAdapter.clear();


    if (bluetoothAdapter == null) {
        if (deviceAdapter != null) deviceAdapter.add("Bluetooth is not supported on this phone");
        if (deviceAdapter != null) deviceAdapter.notifyDataSetChanged();
        return;
    }
```

```
    if (!hasBluetoothPermission()) {
        requestRequiredPermissions();
        if (deviceAdapter != null) deviceAdapter.add("Bluetooth permission is required");
        if (deviceAdapter != null) deviceAdapter.notifyDataSetChanged();
        return;
    }


    try {
        Set<BluetoothDevice> bonded = bluetoothAdapter.getBondedDevices();
        if (bonded == null || bonded.isEmpty()) {
            deviceAdapter.add("No paired devices found");
        } else {
            for (BluetoothDevice device : bonded) {
                pairedDevices.add(device);
                String name = safeDeviceName(device);
                String mac = device.getAddress() == null ? "" : device.getAddress();
                deviceAdapter.add(name + "\n" + mac);
            }
            int preferred = findPreferredDeviceIndex();
            deviceSpinner.setSelection(preferred);
        }
    } catch (SecurityException ex) {
        deviceAdapter.add("Bluetooth permission is required");
    }
    deviceAdapter.notifyDataSetChanged();
}


private int findPreferredDeviceIndex() {
    for (int i = 0; i < pairedDevices.size(); i++) {
        String name = safeDeviceName(pairedDevices.get(i)).toUpperCase(Locale.US);
        if (name.contains("HC-05") || name.contains("HC05") || name.contains("ECOGUARD")) return i;
    }
    return 0;
}


@SuppressLint("MissingPermission")
private void connectSelectedDevice() {
    if (bluetoothAdapter == null) {
        toast("Bluetooth is not supported on this phone.");
        return;
    }
    if (!hasBluetoothPermission()) {
        requestRequiredPermissions();
        toast("Please allow Bluetooth permission, then try again.");
        return;
    }
    if (pairedDevices.isEmpty()) {
        toast("Pair with HC-05 first, then press Refresh Devices.");
        return;
    }


    int selectedIndex = deviceSpinner.getSelectedItemPosition();
    if (selectedIndex < 0 || selectedIndex >= pairedDevices.size()) {
        toast("Select the HC-05 device first.");
        return;
    }


    BluetoothDevice selectedDevice = pairedDevices.get(selectedIndex);
    setBluetoothUi("Connecting...", ORANGE);
    connectButton.setEnabled(false);


    new Thread(() -> {
        try {
            if (bluetoothAdapter.isDiscovering()) bluetoothAdapter.cancelDiscovery();
            BluetoothSocket socket = selectedDevice.createRfcommSocketToServiceRecord(SPP_UUID);
            socket.connect();
            bluetoothSocket = socket;
            inputStream = socket.getInputStream();
            outputStream = socket.getOutputStream();
            connected = true;
            startReaderThread();
            runOnUiThread(() -> {
                connectButton.setEnabled(true);
                connectButton.setText("Disconnect");
                setBluetoothUi("Connected", GREEN);
                toast("Connected to " + safeDeviceName(selectedDevice));
                appendLog("Connected to " + safeDeviceName(selectedDevice));
                sendCommand("S");
                mainHandler.removeCallbacks(statusPollRunnable);
                mainHandler.postDelayed(statusPollRunnable, 3000L);
            });
        } catch (Exception ex) {
            closeSocketQuietly();
            connected = false;
            runOnUiThread(() -> {
                connectButton.setEnabled(true);
                connectButton.setText("Connect");
                setBluetoothUi("Connection failed", RED);
                toast("Connection failed. Make sure HC-05 is powered and paired.");
                appendLog("Connection error: " + ex.getMessage());
            });
        }
    }).start();
}


private void startReaderThread() {
    keepReading = true;
    readerThread = new Thread(() -> {
        byte[] buffer = new byte[256];
        StringBuilder line = new StringBuilder();
        while (keepReading && connected && inputStream != null) {
            try {
                int count = inputStream.read(buffer);
                if (count <= 0) continue;
                for (int i = 0; i < count; i++) {
                    char c = (char) buffer[i];
                    if (c == '\n' || c == '\r') {
                        if (line.length() > 0) {
                            String message = line.toString().trim();
                            line.setLength(0);
                            handleIncomingMessage(message);
                        }
                    } else {
                        line.append(c);
                        if (line.length() > 500) line.setLength(0);
                    }
                }
            } catch (IOException ex) {
                if (keepReading) handleDisconnected("Bluetooth connection lost.");
```

```
                break;
            }
        }
    });
    readerThread.start();
}


private void handleIncomingMessage(String message) {
    runOnUiThread(() -> {
        appendLog("RX: " + message);
        if (message.startsWith("STATUS") || message.startsWith("ALERT")) {
            parseStatusMessage(message);
        } else if (message.startsWith("SAFE")) {
            parseStatusMessage(message);
            updateFireStatus(false);
            updateGasStatus(false);
            setAlarmUi(false);
            emergencyActive = false;
            stopLocalAlarmSound();
        } else if (message.startsWith("READY")) {
            appendLog("EcoGuard Home controller is ready.");
        } else if (message.startsWith("ACK")) {
            parseStatusMessage(message);
        }
    });
}


private void parseStatusMessage(String message) {
    Map<String, String> map = parseKeyValueMessage(message);


    String fireValue = firstValue(map, "FIRE", "FLAME");
    String gasValue = firstValue(map, "GAS");
    String smartValue = firstValue(map, "SMART");
    String alarmValue = firstValue(map, "ALARM");
    String gasAdc = firstValue(map, "GAS_ADC", "GAS_VALUE");
    String flameAdc = firstValue(map, "FLAME_ADC", "FLAME_VALUE");
    String temperature = firstValue(map, "TEMP", "TEMPERATURE", "DHT_TEMP");
    String humidity = firstValue(map, "HUM", "HUMIDITY", "DHT_HUM");


    boolean hasFire = !fireValue.isEmpty();
    boolean hasGas = !gasValue.isEmpty();
    boolean fire = isOne(fireValue);
    boolean gas = isOne(gasValue);


    if (hasFire) updateFireStatus(fire);
    if (hasGas) updateGasStatus(gas);
    if (!smartValue.isEmpty()) setStatusCard(smartCard, "Smart Mode", isOne(smartValue) ? "ON" : "OFF", isOne(smartValue) ? GREEN : MUTED);
    if (!alarmValue.isEmpty()) setAlarmUi(isOne(alarmValue));


    if (!gasAdc.isEmpty() || !flameAdc.isEmpty()) {
        String current = sensorReadings.getText().toString();
        String previousGas = extractDisplayedValue(current, "Gas ADC: ", "   |   Flame ADC:");
        String previousFlame = extractDisplayedValue(current, "Flame ADC: ", "");
        if (gasAdc.isEmpty()) gasAdc = previousGas.isEmpty() ? "--" : previousGas;
        if (flameAdc.isEmpty()) flameAdc = previousFlame.isEmpty() ? "--" : previousFlame;
        sensorReadings.setText("Gas ADC: " + gasAdc + "   |   Flame ADC: " + flameAdc);
    }


    if (!temperature.isEmpty()) updateTemperatureUi(temperature);
    if (!humidity.isEmpty()) updateHumidityUi(humidity);


    String relayString = firstValue(map, "RELAYS");
    if (!relayString.isEmpty()) {
        applyRelayString(relayString);
    } else {
        applyRelayValues(map);
    }


    if (message.startsWith("ALERT") || hasFire || hasGas) {
        evaluateEmergency(fire, gas, message);
    }
}


private Map<String, String> parseKeyValueMessage(String message) {
    Map<String, String> map = new HashMap<>();
    String[] parts = message.split("[;,]");
    for (String part : parts) {
        int equals = part.indexOf('=');
        if (equals > 0 && equals < part.length() - 1) {
            String key = part.substring(0, equals).trim().toUpperCase(Locale.US);
            String value = part.substring(equals + 1).trim();
            map.put(key, value);
        }
    }
    return map;
}


private String firstValue(Map<String, String> map, String... keys) {
    for (String key : keys) {
        String value = map.get(key.toUpperCase(Locale.US));
        if (value != null) return value;
    }
    return "";
}


private void applyRelayString(String relays) {
    long now = System.currentTimeMillis();
    for (int i = 0; i < relays.length() && i < relayStates.length; i++) {
        char c = relays.charAt(i);
        boolean remoteState = c == '1';
        applyRemoteRelayState(i, remoteState, now);
    }
}


private void applyRelayValues(Map<String, String> map) {
    long now = System.currentTimeMillis();
    for (int i = 0; i < relayStates.length; i++) {
        String relayValue = firstValue(map, "R" + (i + 1));
        if (!relayValue.isEmpty()) {
            applyRemoteRelayState(i, isOne(relayValue), now);
        }
    }
```

```
    }


    private void applyRemoteRelayState(int index, boolean remoteState, long now) {
        if (now < pendingRelayUntil[index]) {
            setRelayUi(index, pendingRelayStates[index]);
            return;
        }
        pendingRelayUntil[index] = 0L;
        setRelayUi(index, remoteState);
    }


    private void updateTemperatureUi(String value) {
        if (isErrorValue(value)) {
            setStatusCard(temperatureCard, "Temperature", "ERR", ORANGE);
        } else {
            setStatusCard(temperatureCard, "Temperature", value + " °C", TEAL);
        }
    }


    private void updateHumidityUi(String value) {
        if (isErrorValue(value)) {
            setStatusCard(humidityCard, "Humidity", "ERR", ORANGE);
        } else {
            setStatusCard(humidityCard, "Humidity", value + " %", TEAL);
        }
    }


    private boolean isErrorValue(String value) {
        return value == null || value.trim().isEmpty() || value.equalsIgnoreCase("ERR") || value.equalsIgnoreCase("NAN");
    }


    private String extractDisplayedValue(String text, String startToken, String endToken) {
        int start = text.indexOf(startToken);
        if (start < 0) return "";
        start += startToken.length();
        int end = endToken.isEmpty() ? text.length() : text.indexOf(endToken, start);
        if (end < 0) end = text.length();
        return text.substring(start, end).trim();
    }


    private void updateFireStatus(boolean fireDetected) {
        setStatusCard(fireCard, "Fire", fireDetected ? "ALERT" : "SAFE", fireDetected ? RED : GREEN);
    }


    private void updateGasStatus(boolean gasDetected) {
        setStatusCard(gasCard, "Gas", gasDetected ? "ALERT" : "SAFE", gasDetected ? ORANGE : GREEN);
    }


    private void setAlarmUi(boolean active) {
        setStatusCard(alarmCard, "Alarm", active ? "ACTIVE" : "OFF", active ? RED : MUTED);
    }


    private void evaluateEmergency(boolean fire, boolean gas, String rawMessage) {
        if (fire || gas) {
            long now = System.currentTimeMillis();
            boolean shouldNotify = !emergencyActive || now - lastEmergencyNotificationTime > 15000L;
            emergencyActive = true;
            if (shouldNotify) {
                lastEmergencyNotificationTime = now;
                triggerEmergencyAlert(fire, gas, rawMessage);
            }
        } else {
            emergencyActive = false;
        }
    }


    private boolean isOne(String value) {
        return value != null && (value.equals("1") || value.equalsIgnoreCase("true") || value.equalsIgnoreCase("ON") ||
value.equalsIgnoreCase("ALERT"));
    }


    private boolean sendCommand(String command) {
        return sendCommand(command, true);
    }


    private boolean sendCommand(String command, boolean showInLog) {
        if (command == null || command.trim().isEmpty()) {
            toast("Command is empty. Check Settings.");
            return false;
        }
        final String cleanCommand = command.trim();
        if (!connected || outputStream == null) {
            toast("Connect to HC-05 first.");
            if (showInLog) appendLog("Cannot send '" + cleanCommand + "': not connected.");
            return false;
        }
        new Thread(() -> {
            try {
                synchronized (writeLock) {
                    outputStream.write(cleanCommand.getBytes(StandardCharsets.US_ASCII));
                    outputStream.flush();
                }
                if (showInLog) appendLog("TX: " + cleanCommand);
            } catch (IOException ex) {
                handleDisconnected("Command failed. Bluetooth connection may be lost.");
            }
        }).start();
        return true;
    }


    private void disconnect() {
        keepReading = false;
        connected = false;
        mainHandler.removeCallbacks(statusPollRunnable);
        stopLocalAlarmSound();
        closeSocketQuietly();
        runOnUiThread(() -> {
            if (connectButton != null) {
                connectButton.setText("Connect");
                connectButton.setEnabled(true);
            }
            setBluetoothUi("Disconnected", BLUE);
```

```
        appendLog("Disconnected.");
        toast("Disconnected");
    });
}


private void handleDisconnected(String message) {
    keepReading = false;
    connected = false;
    mainHandler.removeCallbacks(statusPollRunnable);
    closeSocketQuietly();
    runOnUiThread(() -> {
        if (connectButton != null) {
            connectButton.setText("Connect");
            connectButton.setEnabled(true);
        }
        setBluetoothUi("Disconnected", RED);
        appendLog(message);
        toast(message);
    });
}


private void closeSocketQuietly() {
    try { if (inputStream != null) inputStream.close(); } catch (Exception ignored) { }
    try { if (outputStream != null) outputStream.close(); } catch (Exception ignored) { }
    try { if (bluetoothSocket != null) bluetoothSocket.close(); } catch (Exception ignored) { }
    inputStream = null;
    outputStream = null;
    bluetoothSocket = null;
}


@Override
protected void onDestroy() {
    disconnect();
    super.onDestroy();
}


private void triggerEmergencyAlert(boolean fire, boolean gas, String rawMessage) {
    String title;
    String text;
    int notificationId;
    if (fire && gas) {
        title = "EcoGuard EMERGENCY";
        text = "Fire and gas danger detected. All loads should be OFF. Check the area immediately.";
        notificationId = 3003;
    } else if (fire) {
        title = "EcoGuard FIRE Alert";
        text = "Flame/fire sensor detected danger. Check the area immediately.";
        notificationId = 3001;
    } else {
        title = "EcoGuard GAS Alert";
        text = "Gas sensor detected abnormal level. Ventilate the area and check immediately.";
        notificationId = 3002;
    }
    toast(title);
    vibratePhone();
    playLocalAlarmSound();
    showNotification(title, text + "\n" + rawMessage, notificationId);
}


private void showNotification(String title, String text, int notificationId) {
    if (Build.VERSION.SDK_INT >= 33 && checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS) != PackageManager.PERMISSION_GRANTED) {
        requestRequiredPermissions();
        return;
    }
    NotificationManager manager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);
    Intent intent = new Intent(this, MainActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    int flags = PendingIntent.FLAG_UPDATE_CURRENT;
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) flags |= PendingIntent.FLAG_IMMUTABLE;
    PendingIntent pendingIntent = PendingIntent.getActivity(this, notificationId, intent, flags);


    Notification.Builder builder = Build.VERSION.SDK_INT >= Build.VERSION_CODES.O
            ? new Notification.Builder(this, CHANNEL_ID)
            : new Notification.Builder(this);


    builder.setSmallIcon(android.R.drawable.ic_dialog_alert)
            .setContentTitle(title)
            .setContentText(text)
            .setStyle(new Notification.BigTextStyle().bigText(text))
            .setContentIntent(pendingIntent)
            .setAutoCancel(true)
            .setPriority(Notification.PRIORITY_MAX)
            .setCategory(Notification.CATEGORY_ALARM)
            .setVisibility(Notification.VISIBILITY_PUBLIC)
            .setVibrate(new long[]{0, 350, 120, 350, 120, 700});


    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) {
        Uri sound = defaultAlarmUri();
        builder.setSound(sound);
    }


    if (manager != null) manager.notify(notificationId, builder.build());
}


private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationManager manager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);
        NotificationChannel channel = new NotificationChannel(
                CHANNEL_ID,
                "EcoGuard Emergency Alerts",
                NotificationManager.IMPORTANCE_HIGH
        );
        channel.setDescription("Fire and gas emergency alerts from EcoGuard Home");
        channel.enableVibration(true);
        channel.setVibrationPattern(new long[]{0, 350, 120, 350, 120, 700});
        channel.setLockscreenVisibility(Notification.VISIBILITY_PUBLIC);
        AudioAttributes attributes = new AudioAttributes.Builder()
                .setUsage(AudioAttributes.USAGE_ALARM)
                .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
                .build();
        channel.setSound(defaultAlarmUri(), attributes);
        if (manager != null) manager.createNotificationChannel(channel);
    }
}
```

```
    private Uri defaultAlarmUri() {
        Uri sound = RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM);
        if (sound == null) sound = RingtoneManager.getDefaultUri(RingtoneManager.TYPE_NOTIFICATION);
        return sound;
    }


    private void playLocalAlarmSound() {
        stopLocalAlarmSound();
        try {
            localAlarmRingtone = RingtoneManager.getRingtone(getApplicationContext(), defaultAlarmUri());
            if (localAlarmRingtone != null) {
                if (Build.VERSION.SDK_INT >= 28) localAlarmRingtone.setLooping(true);
                localAlarmRingtone.play();
                mainHandler.postDelayed(this::stopLocalAlarmSound, 10000L);
            }
        } catch (Exception ignored) { }
    }


    private void stopLocalAlarmSound() {
        try {
            if (localAlarmRingtone != null) {
                if (Build.VERSION.SDK_INT >= 28) {
                    if (localAlarmRingtone.isPlaying()) localAlarmRingtone.stop();
                } else {
                    localAlarmRingtone.stop();
                }
            }
        } catch (Exception ignored) { }
        localAlarmRingtone = null;
    }


    private void vibratePhone() {
        Vibrator vibrator = (Vibrator) getSystemService(Context.VIBRATOR_SERVICE);
        if (vibrator == null) return;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            vibrator.vibrate(VibrationEffect.createWaveform(new long[]{0, 300, 120, 300, 120, 700}, -1));
        } else {
            vibrator.vibrate(new long[]{0, 300, 120, 300, 120, 700}, -1);
        }
    }


    private boolean hasBluetoothPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            return checkSelfPermission(Manifest.permission.BLUETOOTH_CONNECT) == PackageManager.PERMISSION_GRANTED;
        }
        return true;
    }


    private void requestRequiredPermissions() {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.M) return;
        List<String> permissions = new ArrayList<>();
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            if (checkSelfPermission(Manifest.permission.BLUETOOTH_CONNECT) != PackageManager.PERMISSION_GRANTED)
permissions.add(Manifest.permission.BLUETOOTH_CONNECT);
            if (checkSelfPermission(Manifest.permission.BLUETOOTH_SCAN) != PackageManager.PERMISSION_GRANTED)
permissions.add(Manifest.permission.BLUETOOTH_SCAN);
        } else {
            if (checkSelfPermission(Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED)
permissions.add(Manifest.permission.ACCESS_FINE_LOCATION);
        }
        if (Build.VERSION.SDK_INT >= 33 && checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS) != PackageManager.PERMISSION_GRANTED) {
            permissions.add(Manifest.permission.POST_NOTIFICATIONS);
        }
        if (!permissions.isEmpty()) {
            requestPermissions(permissions.toArray(new String[0]), 2001);
        }
    }


    private String safeDeviceName(BluetoothDevice device) {
        try {
            String name = device.getName();
            return name == null ? "HC-05" : name;
        } catch (SecurityException ex) {
            return "HC-05";
        }
    }


    private void setBluetoothUi(String state, int color) {
        setStatusCard(bluetoothCard, "Bluetooth", state, color);
    }


    private TextView sectionTitle(String text) {
        TextView title = new TextView(this);
        title.setText(text);
        title.setTextColor(BLUE_DARK);
        title.setTextSize(20);
        title.setTypeface(Typeface.DEFAULT_BOLD);
        title.setPadding(dp(2), dp(8), dp(2), dp(8));
        return title;
    }


    private LinearLayout cardContainer() {
        LinearLayout layout = new LinearLayout(this);
        layout.setOrientation(LinearLayout.VERTICAL);
        layout.setPadding(dp(14), dp(14), dp(14), dp(14));
        layout.setBackground(roundedStroke(CARD, Color.rgb(221, 229, 238), dp(18)));
        return layout;
    }


    private LinearLayout row() {
        LinearLayout row = new LinearLayout(this);
        row.setOrientation(LinearLayout.HORIZONTAL);
        row.setGravity(Gravity.CENTER);
        return row;
    }


    private Button primaryButton(String text, int color) {
        Button button = new Button(this);
        button.setText(text);
        button.setTextColor(Color.WHITE);
        button.setTextSize(13);
        button.setAllCaps(false);
```

```
        button.setTypeface(Typeface.DEFAULT_BOLD);
        button.setBackground(roundedFill(color, dp(12)));
        return button;
    }


    private Button outlineButton(String text) {
        Button button = new Button(this);
        button.setText(text);
        button.setTextColor(BLUE_DARK);
        button.setTextSize(13);
        button.setAllCaps(false);
        button.setTypeface(Typeface.DEFAULT_BOLD);
        button.setBackground(roundedStroke(Color.WHITE, BLUE, dp(12)));
        return button;
    }


    private TextView statusCard(String label, String value, int accentColor) {
        TextView textView = new TextView(this);
        textView.setGravity(Gravity.CENTER);
        textView.setTextSize(15);
        textView.setTypeface(Typeface.DEFAULT_BOLD);
        textView.setPadding(dp(10), dp(14), dp(10), dp(14));
        setStatusCard(textView, label, value, accentColor);
        return textView;
    }


    private void setStatusCard(TextView view, String label, String value, int accentColor) {
        if (view == null) return;
        view.setText(label + "\n" + value);
        view.setTextColor(accentColor);
        view.setBackground(roundedStroke(Color.WHITE, accentColor, dp(16)));
    }


    private GradientDrawable roundedFill(int color, int radius) {
        GradientDrawable drawable = new GradientDrawable();
        drawable.setColor(color);
        drawable.setCornerRadius(radius);
        return drawable;
    }


    private GradientDrawable roundedStroke(int fill, int stroke, int radius) {
        GradientDrawable drawable = new GradientDrawable();
        drawable.setColor(fill);
        drawable.setStroke(dp(1), stroke);
        drawable.setCornerRadius(radius);
        return drawable;
    }


    private GradientDrawable roundedGradient(int start, int end) {
        GradientDrawable drawable = new GradientDrawable(GradientDrawable.Orientation.LEFT_RIGHT, new int[]{start, end});
        drawable.setCornerRadius(dp(22));
        return drawable;
    }


    private LinearLayout.LayoutParams matchWrapMargins(int left, int top, int right, int bottom) {
        LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT,
LinearLayout.LayoutParams.WRAP_CONTENT);
        params.setMargins(dp(left), dp(top), dp(right), dp(bottom));
        return params;
    }


    private LinearLayout.LayoutParams weightedButtonParams(float weight) {
        LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(0, dp(48), weight);
        params.setMargins(dp(4), 0, dp(4), 0);
        return params;
    }


    private GridLayout.LayoutParams gridParams() {
        GridLayout.LayoutParams params = new GridLayout.LayoutParams();
        params.width = 0;
        params.height = GridLayout.LayoutParams.WRAP_CONTENT;
        params.columnSpec = GridLayout.spec(GridLayout.UNDEFINED, 1f);
        params.setMargins(dp(4), dp(4), dp(4), dp(4));
        return params;
    }


    private void loadControlSettings() {
        SharedPreferences prefs = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
        for (int i = 0; i < 8; i++) {
            relayLabels[i] = prefs.getString("label_" + i, DEFAULT_LABELS[i]);
            relayOnCommands[i] = prefs.getString("on_" + i, DEFAULT_ON_COMMANDS[i]);
            relayOffCommands[i] = prefs.getString("off_" + i, DEFAULT_OFF_COMMANDS[i]);
            if (relayLabels[i] == null || relayLabels[i].trim().isEmpty()) relayLabels[i] = DEFAULT_LABELS[i];
            if (relayOnCommands[i] == null || relayOnCommands[i].trim().isEmpty()) relayOnCommands[i] = DEFAULT_ON_COMMANDS[i];
            if (relayOffCommands[i] == null || relayOffCommands[i].trim().isEmpty()) relayOffCommands[i] = DEFAULT_OFF_COMMANDS[i];
        }
    }


    private void saveControlSettings() {
        SharedPreferences.Editor editor = getSharedPreferences(PREFS_NAME, MODE_PRIVATE).edit();
        for (int i = 0; i < 8; i++) {
            editor.putString("label_" + i, relayLabels[i]);
            editor.putString("on_" + i, relayOnCommands[i]);
            editor.putString("off_" + i, relayOffCommands[i]);
        }
        editor.apply();
    }


    private void resetControlDefaults() {
        for (int i = 0; i < 8; i++) {
            relayLabels[i] = DEFAULT_LABELS[i];
            relayOnCommands[i] = DEFAULT_ON_COMMANDS[i];
            relayOffCommands[i] = DEFAULT_OFF_COMMANDS[i];
        }
        saveControlSettings();
        refreshAllRelayUi();
        toast("Defaults restored.");
    }


    private void refreshAllRelayUi() {
        for (int i = 0; i < relaySwitches.length; i++) {
            setRelayUi(i, relayStates[i]);
```

```
    }
    updateDoorButtons();
}


private void showControlSettingsDialog() {
    ScrollView scroll = new ScrollView(this);
    LinearLayout content = new LinearLayout(this);
    content.setOrientation(LinearLayout.VERTICAL);
    content.setPadding(dp(14), dp(8), dp(14), dp(4));
    scroll.addView(content);


    TextView note = new TextView(this);
    note.setText("Use one ASCII symbol for each ON/OFF command. These symbols must match the Arduino switch-case commands.");
    note.setTextColor(MUTED);
    note.setTextSize(13);
    note.setPadding(0, 0, 0, dp(10));
    content.addView(note);


    EditText[] labelInputs = new EditText[8];
    EditText[] onInputs = new EditText[8];
    EditText[] offInputs = new EditText[8];


    for (int i = 0; i < 8; i++) {
        TextView relayTitle = new TextView(this);
        relayTitle.setText("Relay " + (i + 1));
        relayTitle.setTextColor(BLUE_DARK);
        relayTitle.setTextSize(15);
        relayTitle.setTypeface(Typeface.DEFAULT_BOLD);
        relayTitle.setPadding(0, dp(8), 0, dp(4));
        content.addView(relayTitle);


        labelInputs[i] = editText("Button name", relayLabels[i], false);
        content.addView(labelInputs[i], matchWrapMargins(0, 0, 0, 6));


        LinearLayout commandRow = row();
        onInputs[i] = editText("ON", relayOnCommands[i], true);
        offInputs[i] = editText("OFF", relayOffCommands[i], true);
        commandRow.addView(onInputs[i], weightedButtonParams(1));
        commandRow.addView(offInputs[i], weightedButtonParams(1));
        content.addView(commandRow, matchWrapMargins(0, 0, 0, 6));
    }


    AlertDialog dialog = new AlertDialog.Builder(this)
            .setTitle("Button Names & Commands")
            .setView(scroll)
            .setPositiveButton("Save", null)
            .setNegativeButton("Cancel", null)
            .setNeutralButton("Reset Defaults", null)
            .create();


    dialog.setOnShowListener(d -> {
        dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener(v -> {
            for (int i = 0; i < 8; i++) {
                String label = labelInputs[i].getText().toString().trim();
                String on = onInputs[i].getText().toString().trim();
                String off = offInputs[i].getText().toString().trim();
                if (label.isEmpty()) label = DEFAULT_LABELS[i];
                if (on.isEmpty()) on = DEFAULT_ON_COMMANDS[i];
                if (off.isEmpty()) off = DEFAULT_OFF_COMMANDS[i];
                relayLabels[i] = label;
                relayOnCommands[i] = on.substring(0, 1);
                relayOffCommands[i] = off.substring(0, 1);
            }
            saveControlSettings();
            refreshAllRelayUi();
            toast("Settings saved.");
            dialog.dismiss();
        });
        dialog.getButton(AlertDialog.BUTTON_NEUTRAL).setOnClickListener(v -> {
            resetControlDefaults();
            dialog.dismiss();
        });
    });


    dialog.show();
}


private EditText editText(String hint, String value, boolean oneChar) {
    EditText editText = new EditText(this);
    editText.setHint(hint);
    editText.setText(value);
    editText.setTextSize(14);
    editText.setSingleLine(true);
    editText.setSelectAllOnFocus(true);
    editText.setInputType(InputType.TYPE_CLASS_TEXT | InputType.TYPE_TEXT_FLAG_CAP_CHARACTERS);
    if (oneChar) editText.setFilters(new InputFilter[]{new InputFilter.LengthFilter(1)});
    return editText;
}


private void appendLog(String message) {
    if (Looper.myLooper() != Looper.getMainLooper()) {
        mainHandler.post(() -> appendLog(message));
        return;
    }
    if (lastMessage == null) return;
    String line = String.format(Locale.US, "%tT  %s", System.currentTimeMillis(), message);
    logLines.add(line);
    while (logLines.size() > 8) logLines.remove(0);
    StringBuilder builder = new StringBuilder();
    for (String logLine : logLines) {
        builder.append(logLine).append('\n');
    }
    lastMessage.setText(builder.toString().trim());
}


private void toast(String message) {
    runOnUiThread(() -> Toast.makeText(MainActivity.this, message, Toast.LENGTH_SHORT).show());
}


private int dp(int value) {
    return (int) (value * getResources().getDisplayMetrics().density + 0.5f);
}
```

```
}
```

Appendix I: Android Manifest and Build Configuration

The manifest and Gradle configuration are included to document the application permissions, package identity and build environment. These files are particularly important because Bluetooth and notification behavior depend on Android permission declarations and target SDK behavior.

I.1 AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <uses-permission android:name="android.permission.BLUETOOTH_SCAN" android:usesPermissionFlags="neverForLocation" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.VIBRATE" />


    <uses-feature android:name="android.hardware.bluetooth" android:required="true" />


    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher_ecoguard"
        android:label="EcoGuard Home"
        android:theme="@style/AppTheme"
        android:usesCleartextTraffic="false">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

I.2 app/build.gradle

```
plugins {
    id 'com.android.application'
}

android {
    namespace 'com.ecoguard.home'
    compileSdk 35


    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }


    defaultConfig {
        applicationId 'com.ecoguard.home'
        minSdk 23
        targetSdk 35
        versionCode 2
        versionName '1.1.0-dht'
    }
}
```